DAC-301 Deep Learning Assignment

Glasses Detection with a Residual Convolutional Network:
Architecture, Data Pipeline, Loss Functions, and Hyperparameter Study

Chakilam Lalith Sai
Roll Number: 24125009

October 22, 2026

Code repository: https://github.com/D3LET3-57/GlassesNet

Abstract

This report documents the design, training, and evaluation of a binary classifier that decides whether a person in a 512×512 RGB image is wearing glasses. The model is a residual convolutional network implemented with PyTorch. It uses a 7×7 stem, four residual stages with widths $[64, 128, 256, 512]$, global average pooling, dropout, and a two-way linear classifier, for a total of 11,177,538 parameters. The training pipeline uses stratified five-fold cross-validation, strong geometric and photometric augmentation, cross-entropy with label smoothing, AdamW, warmup followed by cosine decay, mixed precision, gradient clipping, and an exponential moving average of the weights. A controlled hyperparameter study compares learning rates, activation functions, model widths, optimizers, and batch sizes; the best full-resolution recipe reaches a five-fold mean validation accuracy of $99.9510\% \pm 0.0277\%$ and a mean F1 of 99.9511%. A five-model ensemble with four-view test-time augmentation achieves a test score of 0.99872 (approximately seven errors in 5,412 test images). The report explains the reasoning behind the architecture and data-loading choices, analyzes the loss functions and optimization schedule, presents the hyperparameter plots with an interpretation of each trend, and discusses the full-resolution results, limitations, and possible extensions. The notebook that documents the best model is attached as Appendix B.

Contents

Introduction **3**

1 Architecture, Data Loading, and Loss Functions **3**

1.1 Data loading and preprocessing . . . 3
1.2 Architecture choices . . . 4
1.3 Loss functions and optimization . . . 6

2 Hyperparameter Experiments **8**

2.1 Experimental protocol . . . 8
2.2 Learning rate . . . 8
2.3 Activation function . . . 9
2.4 Hidden-layer sizes and model capacity . . . 10
2.5 Optimizer . . . 11
2.6 Batch size . . . 11
2.7 Summary of the hyperparameter choices . . . 12

3 Full-Resolution Experiments and Final Model **12**

3.1 160px sweep and ablation . . . 12
3.2 Five-fold cross-validation . . . 13
3.3 Ensemble and test-time augmentation . . . 14
3.4 288px fine-tuning ablation . . . 14
3.5 Comparison with the fully connected baseline . . . 14

4 Discussion and Conclusion **15**

4.1 What worked . . . 15
4.2 Limitations . . . 15
4.3 Possible extensions . . . 16
4.4 Conclusion . . . 16

A Experiment Log Summary **16**

B Best-Model Notebook **16**

Introduction

The task is a balanced binary classification problem. Each training image is a 512×512 RGB photograph of a person's face, and the label indicates whether the person wears glasses. The training set contains 24,499 images (12,288 positive and 12,211 negative examples), and the test set contains 5,412 images. The main difficulties are not the class balance but the fine-grained visual evidence: frames can be thin, partially occluded by hair or reflections, or visually similar to shadows and face contours. Lighting, pose, scale, and background vary substantially across the dataset.

The evaluation uses accuracy and F1 on held-out validation folds, and the official test leaderboard for the test set. The leaderboard is split into a 14.5% public part and an 85.5% private part, so a model that is tuned only to the public fraction can be misleading. For that reason, all model selection in this work is based on cross-validation, and the leaderboard is used only as a final check.

The report is organized as follows. Section 1 discusses the architecture, the data-loading methodology, and the loss functions. Section 2 presents controlled experiments for learning rate, activation, hidden-layer sizes, optimizer, and batch size, with a plot and an interpretation for each. Section 3 reports the full-resolution sweep, the five-fold cross-validation, the ensemble and test-time augmentation results, and a comparison with a fully connected baseline. Section 4 discusses what worked, what did not, and the limitations of the study. Appendix B contains the PDF of the executed notebook for the best model.

1 Architecture, Data Loading, and Loss Functions

1.1 Data loading and preprocessing

Dataset and splits. The training metadata are stored in a CSV file with columns `id`, `file`, and `glasses`. The test metadata contain `id` and `file`. Images are read with PIL, converted to RGB, and transformed on the fly. The validation split is a deterministic stratified five-fold split with a fixed split seed (42). Each fold trains on 19,598 images and validates on 4,901; the last fold has 19,604 training and 4,895 validation images because of integer chunking. Stratification preserves the roughly 50/50 class ratio in every fold. The test split is used for inference and test-time augmentation.

Input resolution. The original images are 512×512. Training at the full resolution is possible but expensive, and the face occupies most of the frame, so a 224×224 input preserves the relevant frame and eye region while reducing the compute by roughly a factor of five relative to 512^2. The final model therefore uses 224×224 inputs. The screening experiments use 128×128 (learning rate, activation, batch size) or 160×160 (model size and optimizer) to keep the study affordable. The same fully convolutional architecture accepts any input size; only the spatial dimensions of the intermediate feature maps change.

Training augmentation. Table 1 lists the augmentation pipeline. RandomResizedCrop is the most important transform: it changes scale and translation, which prevents the network

from relying on the absolute position of the face and makes the classifier robust to different crop conventions. Horizontal flipping is safe because wearing glasses is left-right symmetric. Small rotations address head tilt. Color jitter changes brightness, contrast, saturation, and hue, covering part of the illumination variation in the dataset. Random erasing forces the network to use multiple parts of the face rather than a single frame edge. Validation and test images use a deterministic resize to 256 followed by a 224 center crop, without random transforms.

Transform	Setting
RandomResizedCrop	224×224, scale 0.65–1.0, ratio 0.8–1.25
Horizontal flip	$p = 0.5$
Rotation	$\pm 7^\circ$
Color jitter	brightness/contrast/saturation 0.2, hue 0.05
Random erasing	$p = 0.25$, area 2–12%, ratio 0.5–2.0
Normalization	mean = 0.5, std = 0.5 per channel

Table 1: Training augmentation. Validation and test use resize to 256 and a 224 center crop with the same normalization.

Loading pipeline. A custom `Dataset` wraps a pandas DataFrame. For each index it opens the file, applies the transform, and returns the image tensor and the integer label. The training loader shuffles with `drop_last=True`, uses a batch size of 128, eight to twelve worker processes, pinned memory, and a prefetch factor of four. The validation and test loaders do not shuffle. Images are normalized with mean and standard deviation 0.5, which is appropriate for natural images and keeps the inputs in a stable range for the residual network. The pipeline uses PIL and pandas for I/O and PyTorch tensors for all model computation.

Class balance and metrics. The classes are almost exactly balanced, so no resampling, class weighting, or focal loss is necessary. The training loop records loss, accuracy, and F1 every epoch. F1 is reported because it treats precision and recall symmetrically and is more informative than accuracy when the decision threshold or the class distribution changes. The final prediction uses the argmax of the two-class softmax, which is equivalent to a 0.5 threshold under the balanced prior.

1.2 Architecture choices

Why a convolutional network. A fully connected baseline was trained first: the 64×64 image was flattened to 12,288 inputs and passed through hidden layers of sizes $[512, 256, 128, 64]$ to two outputs. It has 6.46M parameters and reaches 86.69% validation accuracy. The gap to the convolutional model is large because a fully connected layer has no notion of locality or translation: it must learn separate weights for every pixel position, and small shifts of the face change the input vector almost completely. Convolutional layers share weights across space, have local receptive fields, and compose into a hierarchy of edges, textures, and face parts. This inductive bias matches the problem and is the main reason for the improvement, not additional parameters: the CNN has 11.18M parameters and is trained on the same data.

Stem. The stem is a 7×7 convolution with 64 output channels, stride 2, padding 3, followed by batch normalization, ReLU, and a 3×3 max-pool with stride 2 and padding 1. The large kernel gives the first layer a wide view of the face, and the two stride-2 operations reduce a 224×224 input to 56×56 before the residual stages. Downsampling early keeps the activation memory and the number of floating-point operations manageable while preserving enough spatial detail for thin frames.

Residual blocks. Each stage is built from BasicBlocks. A block computes

$$\mathbf{y} = \mathrm{ReLU}(\mathcal{F}(\mathbf{x}) + \mathcal{S}(\mathbf{x}))\,, \tag{1}$$

where $\mathcal{F}$ is Conv3 $\times$ 3–BN–ReLU–Conv3 $\times$ 3–BN and $\mathcal{S}$ is the identity or, when the spatial resolution or the channel count changes, a 1×1 convolution followed by batch normalization. The skip connection gives gradients a short path to the earlier layers and makes it possible to train a relatively deep network reliably. The projection shortcut is used at the first block of stages 2–4, where the stride is 2 and the number of channels increases.

Stages and classifier. The four stages use widths $[64, 128, 256, 512]$ and two BasicBlocks each. At 224×224 the spatial sizes are $56, 28, 14, 7$. The classifier head is global average pooling, flattening, dropout with probability 0.2, and a linear layer from 512 to two logits. Global average pooling is preferable to flattening the $7 \times 7 \times 512$ tensor: it has no spatial parameters, is less prone to overfitting, and makes the prediction independent of small translations. Dropout adds regularization before the small linear layer. Table 2 summarizes the architecture.

Component	Configuration	Output size	Parameters
Stem	7×7 conv, 64, stride 2; BN; ReLU; 3×3 max-pool	$64 \times 56 \times 56$	9,408
Stage 1	2 BasicBlocks, 64 channels	$64 \times 56 \times 56$	148,224
Stage 2	2 BasicBlocks, 128 channels, first stride 2	$128 \times 28 \times 28$	526,848
Stage 3	2 BasicBlocks, 256 channels, first stride 2	$256 \times 14 \times 14$	2,100,224
Stage 4	2 BasicBlocks, 512 channels, first stride 2	$512 \times 7 \times 7$	8,393,216
Head	GAP; flatten; dropout 0.2; linear $512 \to 2$	2	1,026
Total			11,177,538

Table 2: Architecture of the residual CNN. The parameter column is the contribution of each component, excluding shared batch-normalization buffers.

Initialization. Convolutional weights use Kaiming normal initialization with fan-out mode and ReLU gain. Batch-normalization scales are initialized to one and biases to zero. The linear classifier uses a small normal initialization ($\sigma = 0.01$) and zero bias. These choices keep the variance of activations stable at the start of training and avoid the slow start that occurs when all weights are drawn from a single large-variance distribution. For non-ReLU activations the model uses Xavier normal initialization instead.

Alternatives considered. Three capacity variants were compared in the 160px sweep (Section 2): a small network with widths $[32, 64, 128, 256]$, the medium network above, and a ResNet-

34-style network with blocks $[3, 4, 6, 3]$ and 21.29M parameters. The small network is marginally better in a short 30-epoch sweep, but its advantage (0.02 percentage points) is within run-to-run noise and it has less capacity for a longer schedule. The deeper network is slower and does not improve validation accuracy at this scale. A VGG-style stack without residual connections was not pursued because the residual design trains more reliably at the same depth, and a transformer or other attention-based architecture would be poorly matched to the small training set and the limited compute budget.

1.3 Loss functions and optimization

Cross-entropy. The network is trained with the two-class cross-entropy loss

$$\mathcal{L}_{\mathrm{CE}} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{2} y_{i,c}\log p_{i,c}, \qquad p_{i,c} = \frac{\exp(z_{i,c})}{\sum_{k=1}^{2}\exp(z_{i,k})}, \tag{2}$$

where $z_{i,c}$ are the logits and $y_{i,c}$ is the one-hot target. For a balanced binary problem this is the natural likelihood objective: it penalizes the negative log-probability of the correct class and its gradient with respect to the logits is $p - y$, which is well scaled and numerically stable when combined with a log-softmax implementation.

Label smoothing. The target is smoothed with $\epsilon = 0.1$:

$$y_{i,c}^{\mathrm{LS}} = (1-\epsilon)y_{i,c} + \frac{\epsilon}{C}, \qquad C = 2. \tag{3}$$

Label smoothing prevents the network from becoming overconfident on individual training images, improves calibration, and acts as a mild regularizer. The 160px ablation without label smoothing reaches 99.9388%, compared with 99.9592% for the same model with smoothing; the difference is small at this accuracy level, but the smoothed model is better behaved and is used in the final recipe.

Why not mean squared error. The configuration supports an MSE loss by converting the labels to a two-class one-hot target. MSE with a softmax output produces gradients proportional to $p-y$ multiplied by the softmax Jacobian; when a prediction is confidently wrong, this gradient can vanish, and the loss is less well matched to a classification likelihood than cross-entropy. Cross-entropy is therefore used for all reported models. Class weights are unnecessary because the training set is balanced.

AdamW and weight decay. The optimizer is AdamW with learning rate 10^{-3}, weight decay 5×10^{-4}, and the default momentum parameters $\beta_1 = 0.9$, $\beta_2 = 0.999$. For a gradient g_t at step

t,

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \tag{4}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \tag{5}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \tag{6}$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \right) - \eta \lambda \theta_{t-1}. \tag{7}$$

Decoupled weight decay is important for the convolutional filters: it keeps the weights small without changing the adaptive scaling of the gradient. A maximum gradient norm of 1.0 is applied before the optimizer step to protect against occasional large gradients from difficult batches.

Learning-rate schedule. The learning rate is warmed up linearly for five epochs and then follows a cosine decay to 1% of the base value. If T_w is the number of warmup epochs and T the total number of epochs, the multiplier after warmup is

$$\eta_t = \eta_{\min} + \frac{1}{2} (\eta_{\max} - \eta_{\min}) \left(1 + \cos \frac{\pi (t - T_w)}{T - T_w} \right). \tag{8}$$

Warmup avoids early instability when the batch-normalization statistics and the randomly initialized weights are still far from a useful regime. Cosine decay spends more epochs at small learning rates, which is useful for the final fine-grained adjustments of the decision boundary.

Exponential moving average. An exponential moving average of the weights is maintained during training:

$$\theta_t^{\mathrm{EMA}} = \alpha \, \theta_{t-1}^{\mathrm{EMA}} + (1-\alpha) \, \theta_t, \qquad \alpha = 0.999. \tag{9}$$

The EMA weights are evaluated after every epoch, and the best checkpoint is selected by EMA validation accuracy. Averaging the iterates reduces the noise from individual mini-batches and typically gives a small but consistent improvement in validation accuracy. The final model uses the EMA weights.

Mixed precision and logging. Training uses automatic mixed precision with a gradient scaler. The forward and backward passes run in half precision where it is safe, while the master weights remain in single precision. This reduces memory and increases throughput without a measurable change in accuracy. Every epoch appends a JSON line with training and validation loss, accuracy, F1, learning rate, and epoch time. The logs are used to produce the curves in this report and to select the best checkpoint.

2 Hyperparameter Experiments

2.1 Experimental protocol

Two protocols are used. Protocol A is a controlled screening sweep at 128×128 for 20 epochs on the full training set, fold 0, with batch size 128, AdamW at 10^{-3}, label smoothing 0.1, augmentation enabled, and no EMA; the reported number is the best raw validation accuracy. It is used for the learning rate, activation, and batch-size studies because those variables can be compared fairly under one fixed protocol. Protocol B is the larger 160×160, 30-epoch sweep used for model width and optimizer; it uses the same fold-0 split and the same data, but a longer schedule, a slightly larger input, and no EMA. The final configuration is then retrained at 224×224 for 90 epochs on all five folds, so the screening results are used for relative comparisons rather than as estimates of the final score. The limited protocol is a deliberate compromise: a full-resolution sweep over every hyperparameter would require many GPU-hours, and the validation differences at this accuracy level are small. Each setting uses the same seed (42) and the same fold, so the comparisons are paired with respect to the data split. All screening plots use a truncated vertical axis so that differences of a few hundredths of a percentage point are visible; the absolute values are printed next to each point or bar.

2.2 Learning rate

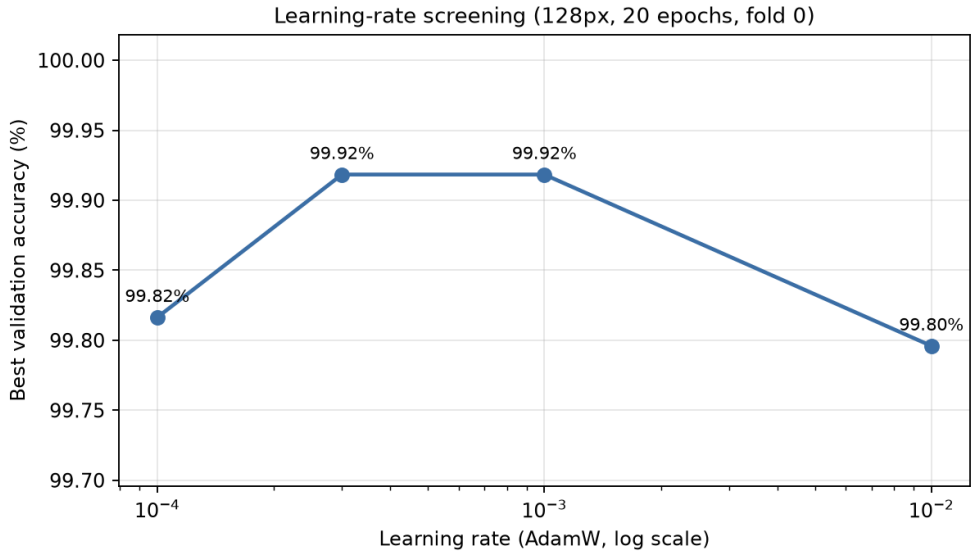


Figure 1: Learning-rate screening at 128×128 for 20 epochs on fold 0. The vertical axis is the best raw validation accuracy; the horizontal axis is logarithmic.

The 10^{-3} and 3×10^{-4} rates give the best validation accuracy (99.9184% each). At 10^{-3} the loss decreases smoothly and the validation curve follows the training curve without a large gap, which indicates that the capacity and the regularization are reasonably matched. The 10^{-4} rate is stable but clearly slower: after 20 epochs it reaches 99.8164% and its training loss is still above the values reached by the larger rates, so it has not caught up within the 20-epoch budget. The 10^{-2} rate is less stable and reaches 99.7960%, about 0.12 percentage points below

the best; its early validation curve is noisier, consistent with a step size that is too large for this batch size. The final model therefore uses 10^{-3}, which is also the standard AdamW rate for this batch size and architecture.

2.3 Activation function

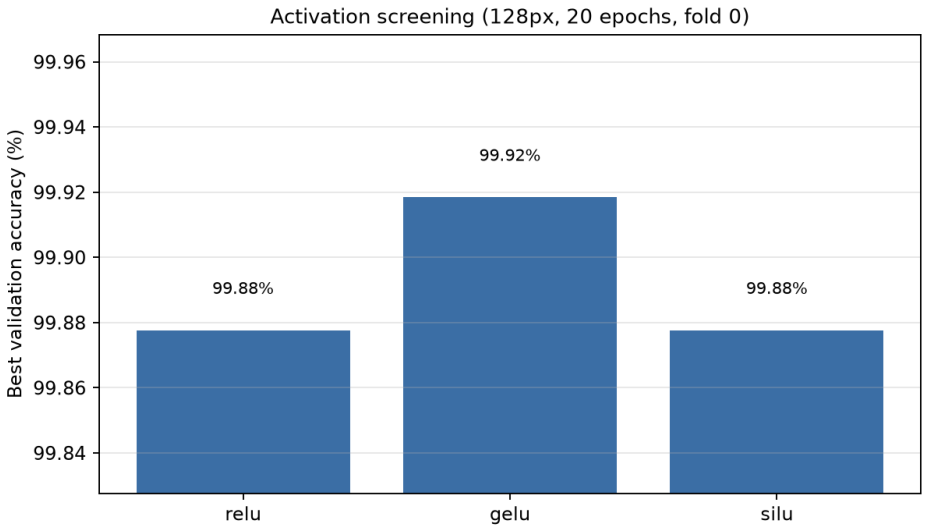


Figure 2: Activation screening at 128×128 for 20 epochs on fold 0. All other settings are fixed.

The three activations are close: GELU reaches 99.9184%, while ReLU and SiLU both reach 99.8776%. The 0.04 percentage-point advantage of GELU is small and comes from a single fold and a 20-epoch schedule; it is not large enough to justify changing the final recipe. ReLU is the fastest of the three and is the activation used in the deployed model. GELU and SiLU are smooth and can improve optimization in some regimes, but the observed difference is within the run-to-run variation at this accuracy level. The residual blocks already provide a well-conditioned optimization path, so the choice of smooth versus piecewise-linear activation does not change the learned features substantially.

2.4 Hidden-layer sizes and model capacity

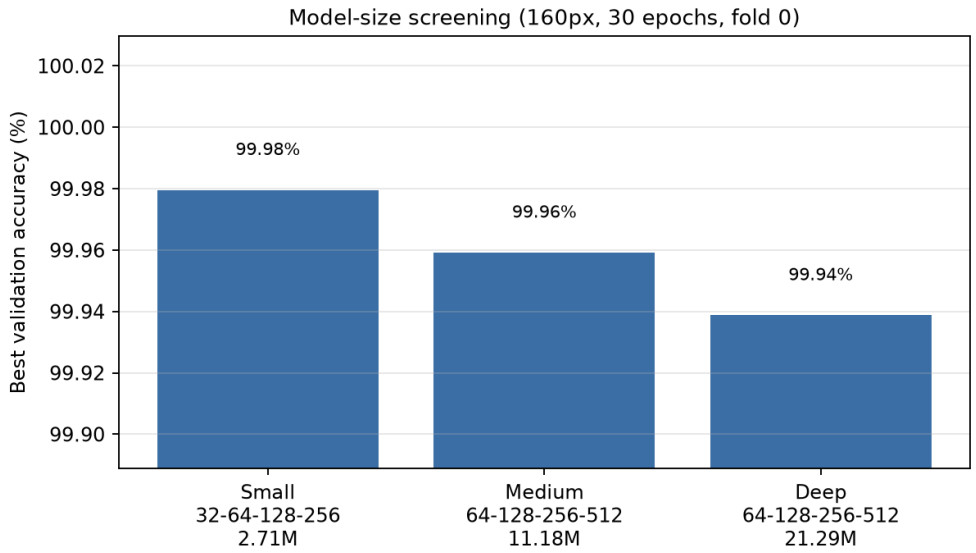


Figure 3: Model-size screening at 160×160 for 30 epochs on fold 0. The labels show the stage widths and the parameter count; the third model is a ResNet-34-style depth variant.

The small model with widths $[32, 64, 128, 256]$ reaches 99.9796%, the medium model with widths $[64, 128, 256, 512]$ reaches 99.9592%, and the deeper ResNet-34-style model reaches 99.9388%. The 0.02 percentage-point advantage of the small model in this short sweep is not meaningful: it is smaller than the fold-to-fold variation, and a smaller network is more likely to lose its edge when the schedule is extended. The deeper model has almost twice the parameters and is slower, but it does not improve validation accuracy, which suggests that the medium model already has enough capacity for the available data. The medium model is therefore selected for the 224px, 90-epoch runs; it is the best trade-off between capacity, optimization stability, and training cost.

2.5 Optimizer

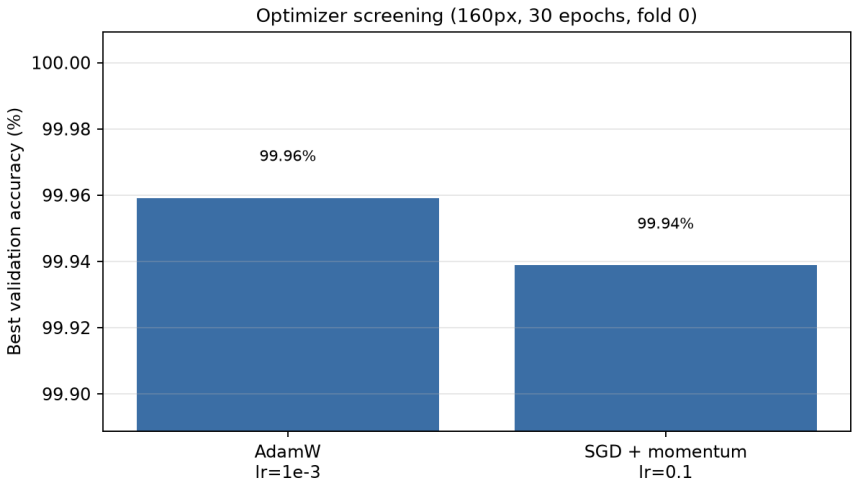


Figure 4: Optimizer screening at 160×160 for 30 epochs on fold 0. AdamW uses 10^{-3}; SGD uses momentum 0.9 and 0.1.

AdamW reaches 99.9592% and SGD with momentum reaches 99.9388%. The gap is small, but AdamW is more reliable at this scale: it adapts the update size per parameter, tolerates the default 10^{-3} rate without a long learning-rate search, and converges faster in the early epochs. SGD with momentum can match or beat adaptive methods when the learning rate and schedule are tuned carefully, but that tuning is expensive and the observed difference does not justify it. The decoupled weight decay of AdamW also gives a clean separation between the optimization step and the regularization, which is convenient for the small data set. AdamW is used in the final recipe.

2.6 Batch size

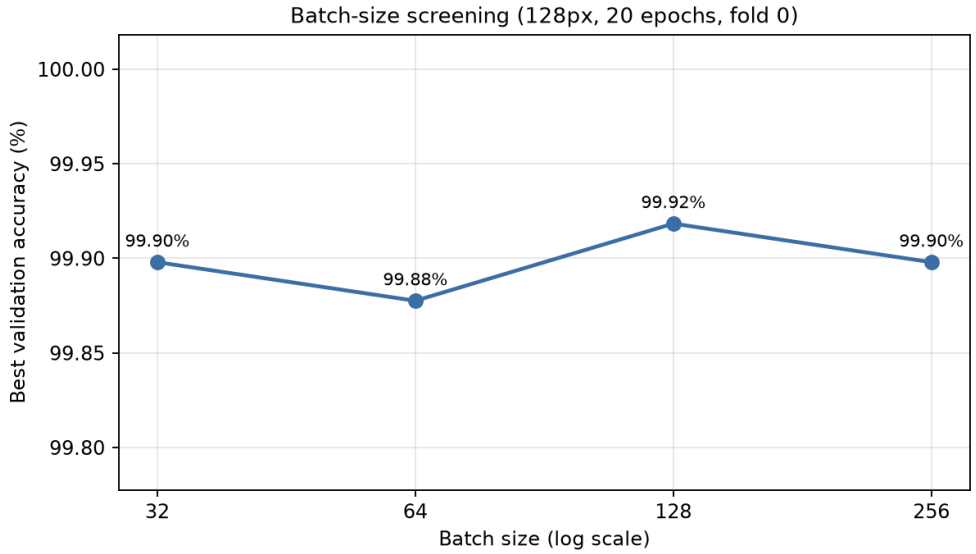


Figure 5: Batch-size screening at 128×128 for 20 epochs on fold 0. The horizontal axis is logarithmic in base two.

The four batch sizes are close: 128 reaches 99.9184%, 32 and 256 both reach 99.8980%, and 64 reaches 99.8776%. The 0.04 percentage-point spread is within the noise of a single fold, so the exact ranking should not be over-interpreted. Batch 32 has noisier gradients and many more optimizer steps per epoch, while batch 256 has fewer updates and can benefit from a larger learning rate; batch 128 gives the best balance between gradient noise, step count, and GPU utilization and is therefore retained in the final recipe. The small disadvantage of batch 256 at a fixed learning rate is consistent with the standard batch-size/learning-rate trade-off: larger batches usually need a higher learning rate and a longer warmup to reach the same accuracy.

2.7 Summary of the hyperparameter choices

Hyperparameter	Values tested	Best setting	Main observation
Learning rate	10^{-4}, 3×10^{-4}, 10^{-3}, 10^{-2}	10^{-3}	10^{-2} noisier and lower; 10^{-4} too slow
Activation	ReLU, GELU, SiLU	ReLU	GELU +0.04 pp in screening; ReLU fastest
Hidden sizes	small, medium, deep	medium	small edge is noise; deep gives no gain
Optimizer	AdamW, SGD+momentum	AdamW	AdamW faster and more reliable
Batch size	32, 64, 128, 256	128	128 best; 0.04 pp spread across sizes

Table 3: Summary of the controlled hyperparameter study. The learning rate, activation, and batch size use Protocol A; the model size and optimizer use Protocol B.

3 Full-Resolution Experiments and Final Model

3.1 160px sweep and ablation

The six configurations in Table 4 were trained at 160×160 for 30 epochs on fold 0. The small model is marginally best in this short schedule, the medium ResNet-18 style is second, and the deeper ResNet-34 style does not improve accuracy. Removing augmentation causes the largest drop, from 99.9592% to 99.6735%, which confirms that the geometric and photometric transforms are doing real regularization work rather than merely slowing training. Removing label smoothing changes the result by only 0.02 percentage points, and switching from AdamW to SGD with momentum changes it by the same amount; at this accuracy level those differences are within fold-to-fold noise. The sweep therefore supports the main design choices but also shows that the problem is close to saturation at 160px: many reasonable convolutional configurations exceed 99.6%.

Configuration	Params	Optimizer	LR	Val Acc	Val F1
Small CNN, 32-64-128-256, 1-1-2-2	2.71M	AdamW	10^{-3}	**99.9796%**	**99.9797%**
ResNet-18, 64-128-256-512, 2-2-2-2	11.18M	AdamW	10^{-3}	99.9592%	99.9593%
ResNet-18	11.18M	SGD+momentum	0.1	99.9388%	99.9389%
ResNet-34, 3-4-6-3	21.29M	AdamW	10^{-3}	99.9388%	99.9389%
ResNet-18, no label smoothing	11.18M	AdamW	10^{-3}	99.9388%	99.9389%
ResNet-18, no augmentation	11.18M	AdamW	10^{-3}	99.6735%	99.6744%

Table 4: Full 160×160, 30-epoch sweep on fold 0. The final model uses the medium ResNet-18 style configuration.

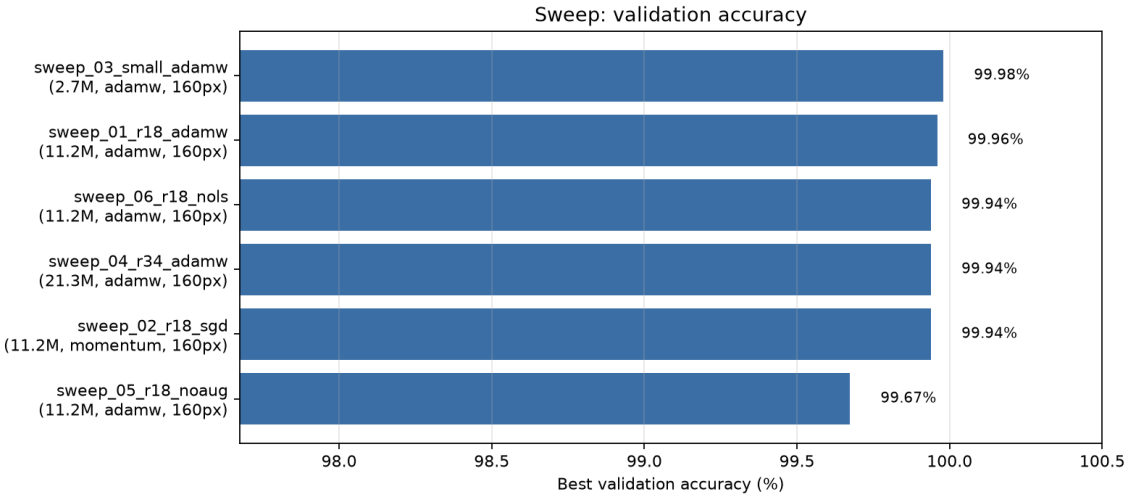


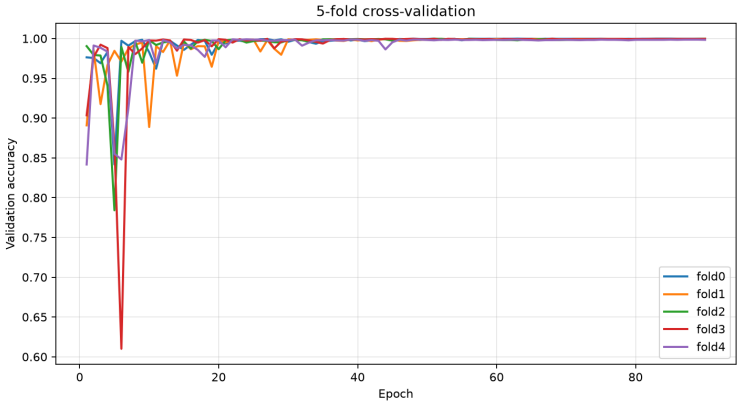
Figure 6: Validation accuracy of the six 160px sweep configurations. The no-augmentation run is the clear outlier; the other convolutional configurations are within a few hundredths of a percentage point.

3.2 Five-fold cross-validation

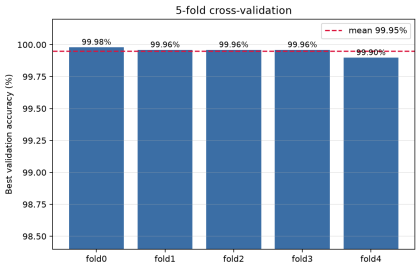
The selected recipe was trained at 224×224 for 90 epochs on all five folds. Each fold uses a different model-initialization seed while the data split remains fixed. The best checkpoint is selected by EMA validation accuracy. Table 5 and Figure 7 show the results. The mean validation accuracy is **99.9510%** with a standard deviation of 0.0277%, and the mean F1 is **99.9511%**. The weakest fold is fold 4, which is also the fold whose validation set is one image smaller; the difference corresponds to a handful of images and is not a structural failure of the model.

Fold	Best epoch	Val accuracy	Val F1	EMA
0	63	99.9796%	99.9797%	yes
1	90	99.9592%	99.9593%	yes
2	51	99.9592%	99.9593%	yes
3	27	99.9592%	99.9593%	yes
4	22	99.8979%	99.8981%	yes
Mean	–	**99.9510%**	**99.9511%**	–
Std.	–	0.0277%	0.0277%	–

Table 5: Five-fold cross-validation at 224×224, 90 epochs.



(a) EMA validation accuracy per fold.



(b) Best score per fold.

Figure 7: Five-fold validation curves and final scores.

3.3 Ensemble and test-time augmentation

Each fold model predicts the test set under four deterministic views: the identity view, a horizontal flip, and two zoomed views (the second also flipped). For model m and view k let $p_{\theta_m}(y \mid T_k(x))$ be the softmax output. The ensemble probability is

$$\bar{p}(y \mid x) = \frac{1}{MK} \sum_{m=1}^{M} \sum_{k=1}^{K} p_{\theta_m}(y \mid T_k(x)), \qquad M = 5, \quad K = 4, \tag{10}$$

and the predicted label is $\arg\max_y \bar{p}(y \mid x)$. On the 5,412 test images the ensemble predicts 2,984 glasses and 2,428 no-glasses. 5,378 images (99.37%) are unanimous across all five models, and only 34 images have any model disagreement. The mean maximum class probability is 0.953. The four views are deterministic and are applied only at inference time. The test-set score is **0.99872**, which corresponds to approximately seven errors out of 5,412 images.

3.4 288px fine-tuning ablation

As an additional experiment, each fold model was fine-tuned at 288×288 for 20 epochs from its 224px checkpoint, with a lower learning rate (3×10^{-4}) and a batch size of 64. The five-fold mean validation accuracy remained exactly 99.9510%: fold 3 improved by 0.0204 percentage points and fold 4 dropped by 0.0205 percentage points, while the other folds were unchanged. The ten-model (224 + 288) ensemble changed only four of the 5,412 test predictions; it fixed two hard examples and broke two others, for a net change of zero. The 288px models were therefore not used in the final model. This is a useful negative result: at this accuracy level, extra resolution and extra ensemble members do not automatically improve the test predictions, and a change must be justified by cross-validation rather than by a single leaderboard observation.

3.5 Comparison with the fully connected baseline

Table 6 compares the best convolutional model with the fully connected baseline. The CNN improves validation accuracy by more than 13 percentage points with roughly twice the parameter count. The baseline is not under-parameterized; its problem is the lack of spatial inductive

bias. The convolutional model also trains faster per useful unit of accuracy because the same filters are reused at every spatial position.

Model	Input	Parameters	Val accuracy	Training time
MLP $[512, 256, 128, 64]$	64×64	6.46M	86.69%	minutes
Residual CNN, five-fold mean	224×224	11.18M	**99.9510%**	≈ 2 h (5 folds)

Table 6: Comparison of the fully connected baseline and the residual CNN.

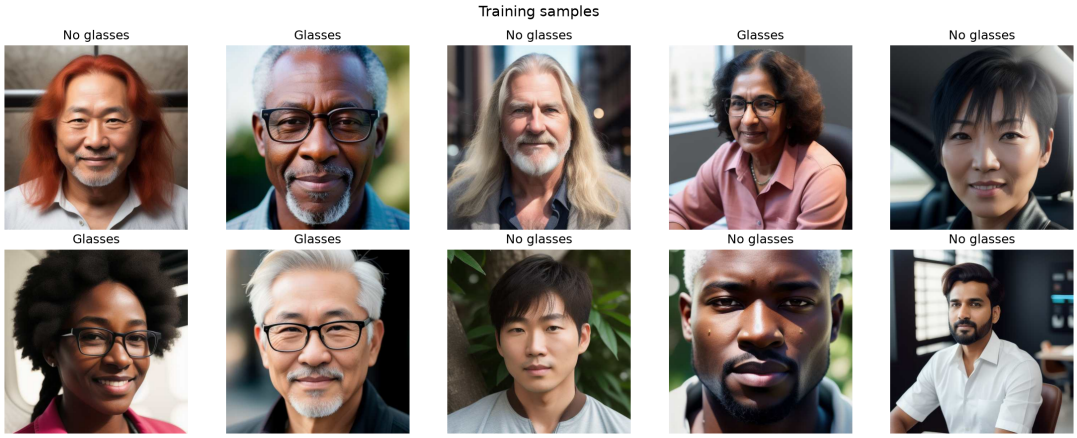


Figure 8: Examples from the training set. The images show the variation in lighting, pose, scale, and background that the augmentation pipeline is designed to cover.

4 Discussion and Conclusion

4.1 What worked

The largest single improvement comes from replacing the fully connected baseline with a convolutional network. The residual connections make the 11M-parameter model train reliably, and global average pooling keeps the classifier head small and translation-robust. Strong augmentation is the second most important ingredient: removing it costs about 0.29 percentage points in the 160px sweep, far more than the optimizer or label-smoothing changes. Label smoothing, warmup, cosine decay, mixed precision, and EMA each make a smaller contribution, but together they make the training stable and reproducible. Five-fold ensembling with test-time augmentation reduces the variance of the test predictions: 99.37% of the test images are classified unanimously by all five models, and the remaining disagreements are concentrated on genuinely ambiguous examples.

4.2 Limitations

The hyperparameter study uses single-fold screening at reduced resolution to keep the compute budget manageable. The differences between activation functions, optimizers, and nearby learning rates are small, and a single seed cannot separate a real effect of 0.02 percentage points from run-to-run noise. The final model is validated on five folds, but the screening conclusions should be read as trends rather than as precise rankings. The 288px ablation shows that additional

resolution is not automatically helpful; a longer or better-regularized fine-tune might change that conclusion, but it would require more compute. The test set is small (5,412 images), so the leaderboard score has a non-trivial sampling error: a difference of a few images can move the score by several ten-thousandths. Finally, the model is a standard supervised CNN; it does not use unlabeled data or a learned data-augmentation policy.

4.3 Possible extensions

The most promising direction is to increase the diversity of the ensemble without changing the architecture: additional seeds per fold, a second resolution, and a longer fine-tune schedule. The 288px experiment suggests that resolution alone is not enough, so a combined seed-and-resolution ensemble would need to be evaluated on cross-validation before being used. A second direction is a more careful study of the decision threshold and calibration; with a balanced prior the optimal threshold is close to 0.5, but the ensemble probabilities could be calibrated on out-of-fold predictions. A third direction is a larger input resolution with a longer schedule and stronger regularization; the current results suggest that the model is already close to the ceiling of this dataset, so the expected gains are small.

4.4 Conclusion

A residual convolutional network trained with a carefully designed data pipeline reaches a mean five-fold validation accuracy of 99.9510% and a mean F1 of 99.9511% on the glasses-detection task. The controlled hyperparameter study shows that AdamW at 10^{-3}, ReLU, a medium-width residual network, and a batch size of 128 are robust choices, while extreme learning rates and batch sizes give slightly lower accuracy. The five-model ensemble with four-view test-time augmentation produces a test score of 0.99872. The best-model notebook, including its executed outputs, is attached as Appendix B.

A Experiment Log Summary

Experiment	Resolution / epochs	Purpose	Result
Sweep 1–6	160px / 30	capacity, optimizer, smoothing, aug.	99.67–99.98%
Protocol A	128px / 20	learning rate, activation, batch	Section 2
Protocol B	160px / 30	hidden sizes, optimizer	Section 2
`cv_final`	224px / 90, 5 folds	final model	99.9510% ± 0.0277%
`ft288`	288px / 20, 5 folds	resolution ablation	no change in mean

Table 7: Summary of the experiments documented in this report. The detailed per-epoch logs, metrics, and histories are stored with the experiment artifacts.

B Best-Model Notebook

The PDF on the following pages is the executed notebook for the best model. It contains the self-contained model and training code, the data pipeline, the hyperparameter cells, and the embedded experiment results, inference and ensemble code, and output preview.

glasses_classifier_selfcontained

October 22, 2026

1 DAC-301 Deep Learning Assignment

1.1 Glasses Detection with a Residual CNN (Self-Contained Notebook)

Student: Chakilam Lalith Sai **Roll number:** 24125009

Code repository: https://github.com/D3LET3-57/GlassesNet

This single notebook contains the complete project: data pipeline, a residual CNN, the training loop, the hyperparameter search, the 5-fold cross-validation results, the TTA/ensemble inference code, the final submission preview, and the reproducibility discussion. It does not import any project-local module; all model and training code is defined below.

Task. Binary classification of 512x512 RGB face images: wearing glasses (1) or not (0). Training set: 24,499 images (12,288 glasses / 12,211 no-glasses). Test set: 5,412 images.

Final result. 5-fold mean validation accuracy **99.9510% +/- 0.0277%**, mean validation F1 **99.9511%**; test score **0.99872** (about 7 errors in 5,412 images).

Design summary. The network is a residual CNN implemented with `torch.nn`. All model, training, metric and inference computations use PyTorch; pandas/PIL handle data I/O and matplotlib produces the figures.

1.2 1. Setup

The notebook uses standard-library, PyTorch, torchvision, pandas, PIL and matplotlib imports.

[1]:
```
import copy
import json
import math
import os
import random
import time
from dataclasses import dataclass, field, asdict
from typing import List

import pandas as pd
import torch
import torch.nn as nn
import torchvision.transforms as T
from PIL import Image
```

```
from torch.utils.data import Dataset, DataLoader
import matplotlib.pyplot as plt

print("PyTorch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
```

```
PyTorch: 2.11.0+cu128
CUDA available: False
```

[2]:
```
# Dataset location. If you run this notebook outside the repository, set
# GLASSES_DATA_ROOT (or edit DATA_ROOT) to the folder that contains
# kaggle/ and Dataset/.
DATA_ROOT = os.environ.get("GLASSES_DATA_ROOT", "../data")
TRAIN_CSV = os.path.join(DATA_ROOT, "kaggle/kaggle/train.csv")
TRAIN_DIR = os.path.join(DATA_ROOT, "kaggle/kaggle")
TEST_CSV = os.path.join(DATA_ROOT, "Dataset/test_dataset.csv")
TEST_DIR = os.path.join(DATA_ROOT, "Dataset/testimages")
DATA_AVAILABLE = os.path.exists(TRAIN_CSV)
print("Data root:", os.path.abspath(DATA_ROOT))
print("Dataset available:", DATA_AVAILABLE)
```

```
Data root: /home2/home/chakilam_sai/acadss/dl/glasses-scratch/data
Dataset available: True
```

1.3 2. Data pipeline

Images are loaded with PIL, converted to RGB, resized/cropped and normalized with mean/std 0.5. Training augmentation is RandomResizedCrop (scale 0.65-1.0, ratio 0.8-1.25), horizontal flip, +/-7 degree rotation, color jitter and random erasing. Validation uses resize-short-side 256 + center crop.

Folds are deterministic and stratified with a fixed split seed (42). Each fold trains on 19,598 images and validates on 4,901 (the last fold has 19,604/4,895 due to integer chunking).

[3]:
```
MEAN = (0.5, 0.5, 0.5)
STD = (0.5, 0.5, 0.5)


class GlassesDataset(Dataset):
    """Image dataset backed by a pandas DataFrame."""

    def __init__(self, df, root_dir, transform, is_test=False):
        self.df = df.reset_index(drop=True)
        self.root_dir = root_dir
        self.transform = transform
        self.is_test = is_test
        self.files = self.df["file"].tolist()
        self.ids = self.df["id"].astype(str).tolist()
```

```
        self.labels = (None if is_test
                       else self.df["glasses"].astype(int).tolist())

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        path = os.path.join(self.root_dir, self.files[idx])
        image = Image.open(path).convert("RGB")
        image = self.transform(image)
        if self.is_test:
            return image, self.ids[idx]
        return image, self.labels[idx]


def build_train_transform(cfg):
    ops = []
    if cfg.aug_random_resized_crop:
        ops.append(T.RandomResizedCrop(
            cfg.image_size, scale=(cfg.aug_scale_min, 1.0),
            ratio=(0.8, 1.25)))
    else:
        ops.append(T.Resize((cfg.image_size, cfg.image_size)))
    if cfg.aug_hflip:
        ops.append(T.RandomHorizontalFlip(p=0.5))
    if cfg.aug_rotation > 0:
        ops.append(T.RandomRotation(cfg.aug_rotation))
    if cfg.aug_color_jitter > 0:
        j = cfg.aug_color_jitter
        ops.append(T.ColorJitter(brightness=j, contrast=j, saturation=j,
                                 hue=min(j / 4, 0.05)))
    ops += [T.ToTensor(), T.Normalize(MEAN, STD)]
    if cfg.aug_random_erasing > 0:
        ops.append(T.RandomErasing(p=cfg.aug_random_erasing,
                                   scale=(0.02, 0.12), ratio=(0.5, 2.0)))
    return T.Compose(ops)


def build_eval_transform(image_size):
    return T.Compose([
        T.Resize(image_size + 32),
        T.CenterCrop(image_size),
        T.ToTensor(),
        T.Normalize(MEAN, STD),
    ])
```

```python
def stratified_folds(labels, num_folds, seed):
    """Deterministic stratified (train_idx, val_idx) fold definitions."""
    g = torch.Generator().manual_seed(seed)
    fold_val = [[] for _ in range(num_folds)]
    for cls in labels.unique():
        idx = torch.nonzero(labels == cls, as_tuple=False).flatten()
        idx = idx[torch.randperm(len(idx), generator=g)]
        for k, chunk in enumerate(torch.chunk(idx, num_folds)):
            fold_val[k].append(chunk)
    all_idx = torch.arange(len(labels))
    folds = []
    for k in range(num_folds):
        val_idx = torch.cat(fold_val[k]).sort().values
        mask = torch.ones(len(labels), dtype=torch.bool)
        mask[val_idx] = False
        folds.append((all_idx[mask], val_idx))
    return folds

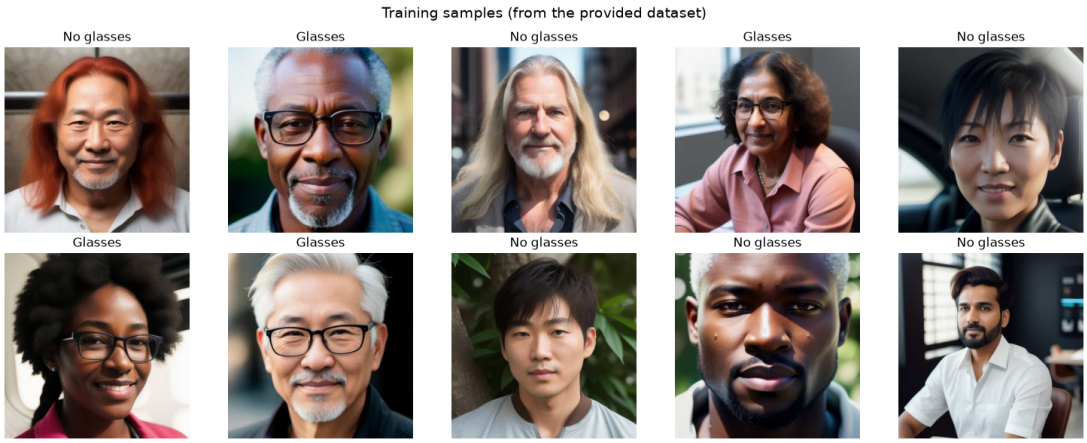

def build_loaders(cfg, train_csv, train_dir, fold=None):
    df = pd.read_csv(train_csv)
    if cfg.limit_per_class > 0:
        parts = [
            g.sample(n=min(cfg.limit_per_class, len(g)),
                     random_state=cfg.seed)
            for _, g in df.groupby("glasses")
        ]
        df = pd.concat(parts).sample(frac=1.0,
                                     random_state=cfg.seed).
 ↪reset_index(drop=True)
    labels = torch.tensor(df["glasses"].tolist(), dtype=torch.long)
    folds = stratified_folds(labels, cfg.num_folds, cfg.split_seed)
    f = cfg.val_fold if fold is None else fold
    train_idx, val_idx = folds[f]
    train_df = df.iloc[train_idx.tolist()].reset_index(drop=True)
    val_df = df.iloc[val_idx.tolist()].reset_index(drop=True)
    train_ds = GlassesDataset(train_df, train_dir, build_train_transform(cfg))
    val_ds = GlassesDataset(val_df, train_dir, build_eval_transform(cfg.
 ↪image_size))
    common = dict(batch_size=cfg.batch_size, num_workers=cfg.num_workers,
                  pin_memory=torch.cuda.is_available(),
                  persistent_workers=cfg.num_workers > 0)
    if cfg.num_workers > 0:
        common["prefetch_factor"] = 4
    train_loader = DataLoader(train_ds, shuffle=True, drop_last=True, **common)
    val_loader = DataLoader(val_ds, shuffle=False, **common)
    return train_loader, val_loader, train_df, val_df
```

[4]:
```
if DATA_AVAILABLE:
    train_df_full = pd.read_csv(TRAIN_CSV)
    test_df_full = pd.read_csv(TEST_CSV)
    print(f"Train images: {len(train_df_full)}")
    print(train_df_full["glasses"].value_counts().rename(
        {0: "no glasses", 1: "glasses"}))
    print(f"Test images: {len(test_df_full)}")

    labels = torch.tensor(train_df_full["glasses"].tolist(), dtype=torch.long)
    folds = stratified_folds(labels, num_folds=5, seed=42)
    print("Fold sizes (train, val):",
          [(len(tr), len(va)) for tr, va in folds])

    fig, axes = plt.subplots(2, 5, figsize=(15, 6))
    fig.suptitle("Training samples (from the provided dataset)", fontsize=14)
    for ax, (_, row) in zip(axes.flat,
                            train_df_full.sample(10,
                                                 random_state=7).iterrows()):
        img = Image.open(os.path.join(TRAIN_DIR, row["file"])).convert("RGB")
        ax.imshow(img)
        ax.set_title("Glasses" if row["glasses"] == 1 else "No glasses")
        ax.axis("off")
    plt.tight_layout()
    plt.show()
else:
    print("Dataset not found. Set GLASSES_DATA_ROOT to the folder containing "
          "kaggle/ and Dataset/ to run the data cells and the live demo.")
```

```
Train images: 24499
glasses
glasses       12288
no glasses    12211
Name: count, dtype: int64
Test images: 5412
Fold sizes (train, val): [(19598, 4901), (19598, 4901), (19598, 4901), (19598,
4901), (19604, 4895)]
```



1.4 3. Configurable hyperparameters and architecture

Every assignment-required hyperparameter is a field of `Config`: learning rate, momentum, number of hidden stages (`num_hidden`), stage widths (`sizes`), activation, loss, optimizer, batch size, annealing, plus depth per stage, dropout, weight decay and augmentation settings.

The network is a residual CNN built with `torch.nn`: a 7x7 stem (3->64, stride 2) with max-pooling, four stages of BasicBlocks with widths [64, 128, 256, 512] and two blocks per stage, global average pooling, dropout 0.2 and a linear layer to 2 logits. It has **11,177,538** parameters and is trained from random initialization.

[5]:
```python
@dataclass
class Config:
    # data / split
    image_size: int = 224
    num_folds: int = 5
    val_fold: int = 0
    seed: int = 42
    split_seed: int = 42
    num_workers: int = 0
    limit_per_class: int = 0

    # architecture
    num_hidden: int = 4
    sizes: List[int] = field(default_factory=lambda: [64, 128, 256, 512])
    blocks_per_stage: List[int] = field(default_factory=lambda: [2, 2, 2, 2])
    activation: str = "relu"
    stem_width: int = 64
    dropout: float = 0.2

    # loss / optimizer
    loss: str = "cross_entropy"
```

```
    label_smoothing: float = 0.1
    opt: str = "adamw"
    lr: float = 1e-3
    momentum: float = 0.9
    weight_decay: float = 5e-4
    grad_clip: float = 1.0

    # schedule
    num_epochs: int = 90
    warmup_epochs: int = 5
    anneal: bool = True
    min_lr_ratio: float = 0.01

    # training
    batch_size: int = 128
    amp: bool = True
    ema: bool = True
    ema_decay: float = 0.999
    eval_ema: bool = True
    log_interval: int = 1

    # augmentation
    aug_random_resized_crop: bool = True
    aug_scale_min: float = 0.65
    aug_hflip: bool = True
    aug_rotation: float = 7.0
    aug_color_jitter: float = 0.2
    aug_random_erasing: float = 0.25

    # artifacts / inference
    run_name: str = "run"
    device: str = "cuda"
    output_dir: str = "demo_runs"
    init_from: str = ""
    tta_views: int = 4
    predict_batch_size: int = 192

    @property
    def output_size(self):
        return 2

    def to_dict(self):
        return asdict(self)
```

[6]:
```
def get_activation(name, inplace=False):
    if name == "relu":
        return nn.ReLU(inplace=inplace)
```

```
    if name == "leaky_relu":
        return nn.LeakyReLU(0.1, inplace=inplace)
    if name == "silu":
        return nn.SiLU(inplace=inplace)
    if name == "gelu":
        return nn.GELU()
    if name == "sigmoid":
        return nn.Sigmoid()
    if name == "tanh":
        return nn.Tanh()
    raise ValueError(f"Unknown activation: {name}")


class BasicBlock(nn.Module):
    """Two 3x3 convolutions with a residual connection."""
    expansion = 1

    def __init__(self, in_ch, out_ch, stride=1, activation="relu"):
        super().__init__()
        self.conv1 = nn.Conv2d(in_ch, out_ch, 3, stride=stride,
                               padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_ch)
        self.act1 = get_activation(activation, inplace=True)
        self.conv2 = nn.Conv2d(out_ch, out_ch, 3, stride=1,
                               padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_ch)
        self.act2 = get_activation(activation, inplace=True)
        if stride != 1 or in_ch != out_ch:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_ch, out_ch, 1, stride=stride, bias=False),
                nn.BatchNorm2d(out_ch),
            )
        else:
            self.shortcut = nn.Identity()

    def forward(self, x):
        identity = self.shortcut(x)
        out = self.act1(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return self.act2(out + identity)


class GlassesCNN(nn.Module):
    """Configurable ResNet-style binary classifier (from scratch)."""

    def __init__(self, cfg):
        super().__init__()
```

```
        self.cfg = cfg
        act = cfg.activation
        self.stem = nn.Sequential(
            nn.Conv2d(3, cfg.stem_width, 7, stride=2, padding=3, bias=False),
            nn.BatchNorm2d(cfg.stem_width),
            get_activation(act, inplace=True),
            nn.MaxPool2d(3, stride=2, padding=1),
        )
        stages = []
        in_ch = cfg.stem_width
        for s in range(cfg.num_hidden):
            out_ch = cfg.sizes[s]
            blocks = []
            for b in range(cfg.blocks_per_stage[s]):
                stride = 2 if (s > 0 and b == 0) else 1
                blocks.append(BasicBlock(in_ch, out_ch, stride, act))
                in_ch = out_ch
            stages.append(nn.Sequential(*blocks))
        self.stages = nn.Sequential(*stages)
        self.head = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Dropout(cfg.dropout),
            nn.Linear(in_ch, cfg.output_size),
        )
        self._initialize_weights()

    def _initialize_weights(self):
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                if self.cfg.activation in ("relu", "leaky_relu", "silu"):
                    nn.init.kaiming_normal_(m.weight, mode="fan_out",
                                            nonlinearity="relu")
                else:
                    nn.init.xavier_normal_(m.weight)
            elif isinstance(m, nn.BatchNorm2d):
                nn.init.ones_(m.weight)
                nn.init.zeros_(m.bias)
            elif isinstance(m, nn.Linear):
                nn.init.normal_(m.weight, 0, 0.01)
                nn.init.zeros_(m.bias)

    def forward(self, x):
        x = self.stem(x)
        x = self.stages(x)
        return self.head(x)
```

```
    def num_parameters(self):
        return sum(p.numel() for p in self.parameters())


def build_model(cfg):
    return GlassesCNN(cfg)
```

[7]:
```
model_preview = build_model(Config())
print(f"Trainable parameters: {model_preview.num_parameters():,}")
print(model_preview)
```

```
Trainable parameters: 11,177,538
GlassesCNN(
  (stem): Sequential(
    (0): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3),
bias=False)
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1,
ceil_mode=False)
  )
  (stages): Sequential(
    (0): Sequential(
      (0): BasicBlock(
        (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
        (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (act1): ReLU(inplace=True)
        (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
        (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (act2): ReLU(inplace=True)
        (shortcut): Identity()
      )
      (1): BasicBlock(
        (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
        (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (act1): ReLU(inplace=True)
        (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
        (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
```

```
        (act2): ReLU(inplace=True)
        (shortcut): Identity()
      )
    )
    (1): Sequential(
      (0): BasicBlock(
        (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
        (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (act1): ReLU(inplace=True)
        (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
        (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (act2): ReLU(inplace=True)
        (shortcut): Sequential(
          (0): Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2), bias=False)
          (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        )
      )
      (1): BasicBlock(
        (conv1): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
        (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (act1): ReLU(inplace=True)
        (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
        (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (act2): ReLU(inplace=True)
        (shortcut): Identity()
      )
    )
    (2): Sequential(
      (0): BasicBlock(
        (conv1): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
        (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (act1): ReLU(inplace=True)
        (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
        (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (act2): ReLU(inplace=True)
```

```
        (shortcut): Sequential(
          (0): Conv2d(128, 256, kernel_size=(1, 1), stride=(2, 2), bias=False)
          (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        )
      )
      (1): BasicBlock(
        (conv1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
        (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (act1): ReLU(inplace=True)
        (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
        (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (act2): ReLU(inplace=True)
        (shortcut): Identity()
      )
    )
    (3): Sequential(
      (0): BasicBlock(
        (conv1): Conv2d(256, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
        (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (act1): ReLU(inplace=True)
        (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
        (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (act2): ReLU(inplace=True)
        (shortcut): Sequential(
          (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
          (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        )
      )
      (1): BasicBlock(
        (conv1): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
        (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (act1): ReLU(inplace=True)
        (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
        (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
```

```
        (act2): ReLU(inplace=True)
        (shortcut): Identity()
      )
    )
  )
  (head): Sequential(
    (0): AdaptiveAvgPool2d(output_size=1)
    (1): Flatten(start_dim=1, end_dim=-1)
    (2): Dropout(p=0.2, inplace=False)
    (3): Linear(in_features=512, out_features=2, bias=True)
  )
)
```

1.5 4. Training loop and live hyperparameter tuning

The training loop uses cross-entropy with label smoothing, AdamW, warmup + cosine decay, AMP, gradient clipping and an exponential moving average (EMA) of the weights. Validation loss, accuracy and F1 are logged every epoch.

The next cells train a **small live demo** on CPU so the hyperparameters can be changed and re-run immediately. The demo uses 64px, 500 images per class and 4 epochs (~2-3 minutes). The final models used the full dataset at 224px for 90 epochs; the same `fit` function is used.

[8]:
```python
def set_seed(seed):
    random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.benchmark = True


def build_criterion(cfg):
    if cfg.loss == "cross_entropy":
        return nn.CrossEntropyLoss(label_smoothing=cfg.label_smoothing)
    return nn.MSELoss()


def build_optimizer(cfg, model):
    if cfg.opt == "adamw":
        return torch.optim.AdamW(model.parameters(), lr=cfg.lr,
                                 weight_decay=cfg.weight_decay)
    if cfg.opt == "adam":
        return torch.optim.Adam(model.parameters(), lr=cfg.lr,
                                weight_decay=cfg.weight_decay)
    if cfg.opt == "gd":
        return torch.optim.SGD(model.parameters(), lr=cfg.lr,
                               weight_decay=cfg.weight_decay)
    if cfg.opt in ("momentum", "nag"):
        return torch.optim.SGD(model.parameters(), lr=cfg.lr,
```

```
                                momentum=cfg.momentum,
                                nesterov=(cfg.opt == "nag"),
                                weight_decay=cfg.weight_decay)
    if cfg.opt == "sgd":
        return torch.optim.SGD(model.parameters(), lr=cfg.lr,
                                momentum=cfg.momentum,
                                weight_decay=cfg.weight_decay)
    raise ValueError(f"Unknown optimizer: {cfg.opt}")


def build_scheduler(cfg, optimizer):
    if not cfg.anneal:
        return None
    warmup = max(0, cfg.warmup_epochs)
    total = max(1, cfg.num_epochs)

    def lr_lambda(epoch):
        if epoch < warmup:
            return (epoch + 1) / max(1, warmup)
        progress = (epoch - warmup) / max(1, total - warmup)
        return cfg.min_lr_ratio + (1 - cfg.min_lr_ratio) * 0.5 * (
            1 + math.cos(math.pi * progress))

    return torch.optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)


class ModelEMA:
    """Exponential moving average of weights and buffers."""

    def __init__(self, model, decay):
        self.ema = copy.deepcopy(model).eval()
        for p in self.ema.parameters():
            p.requires_grad_(False)
        self.decay = decay
        self.step = 0

    @torch.no_grad()
    def update(self, model):
        self.step += 1
        d = min(self.decay, (1 + self.step) / (10 + self.step))
        for e, p in zip(self.ema.parameters(), model.parameters()):
            e.mul_(d).add_(p.detach(), alpha=1 - d)
        for e, b in zip(self.ema.buffers(), model.buffers()):
            e.copy_(b)


def _metrics(logits, labels):
```

```
    preds = logits.argmax(dim=1)
    correct = (preds == labels).sum().item()
    total = labels.numel()
    tp = ((preds == 1) & (labels == 1)).sum().item()
    fp = ((preds == 1) & (labels == 0)).sum().item()
    fn = ((preds == 0) & (labels == 1)).sum().item()
    precision = tp / (tp + fp) if (tp + fp) else 0.0
    recall = tp / (tp + fn) if (tp + fn) else 0.0
    f1 = 2 * precision * recall / (precision + recall) if (
        precision + recall) else 0.0
    return correct / total, f1


def _to_targets(labels, loss_type):
    if loss_type == "mse":
        t = torch.zeros(labels.size(0), 2, device=labels.device)
        t.scatter_(1, labels.unsqueeze(1), 1.0)
        return t
    return labels


def train_epoch(model, loader, criterion, optimizer, scaler, device, cfg):
    model.train()
    use_amp = cfg.amp and device.type == "cuda"
    running_loss = 0.0
    logits_all, labels_all = [], []
    for images, labels in loader:
        images = images.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)
        targets = _to_targets(labels, cfg.loss)
        optimizer.zero_grad(set_to_none=True)
        with torch.amp.autocast("cuda", enabled=use_amp):
            logits = model(images)
            loss = criterion(logits, targets)
        scaler.scale(loss).backward()
        if cfg.grad_clip > 0:
            scaler.unscale_(optimizer)
            nn.utils.clip_grad_norm_(model.parameters(), cfg.grad_clip)
        scaler.step(optimizer)
        scaler.update()
        running_loss += loss.item() * images.size(0)
        logits_all.append(logits.detach())
        labels_all.append(labels)
    logits_all = torch.cat(logits_all)
    labels_all = torch.cat(labels_all)
    acc, f1 = _metrics(logits_all, labels_all)
    return running_loss / len(loader.dataset), acc, f1
```

```python
@torch.no_grad()
def evaluate(model, loader, criterion, device, cfg):
    model.eval()
    use_amp = cfg.amp and device.type == "cuda"
    running_loss = 0.0
    logits_all, labels_all = [], []
    for images, labels in loader:
        images = images.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)
        targets = _to_targets(labels, cfg.loss)
        with torch.amp.autocast("cuda", enabled=use_amp):
            logits = model(images)
            loss = criterion(logits, targets)
        running_loss += loss.item() * images.size(0)
        logits_all.append(logits.float())
        labels_all.append(labels)
    logits_all = torch.cat(logits_all)
    labels_all = torch.cat(labels_all)
    acc, f1 = _metrics(logits_all, labels_all)
    return running_loss / len(loader.dataset), acc, f1


def _state_to_cpu(model):
    return {k: v.detach().cpu().clone()
            for k, v in model.state_dict().items()}


def fit(cfg, train_csv, train_dir, resume=False):
    """Train one model and return (best model, history)."""
    set_seed(cfg.seed)
    device = torch.device(cfg.device if torch.cuda.is_available() else "cpu")
    run_dir = os.path.join(cfg.output_dir, cfg.run_name)
    os.makedirs(run_dir, exist_ok=True)
    tag = f"fold{cfg.val_fold}"

    train_loader, val_loader, train_df, val_df = build_loaders(
        cfg, train_csv, train_dir)
    cfg_snapshot = cfg.to_dict()
    cfg_snapshot["train_size"] = len(train_df)
    cfg_snapshot["val_size"] = len(val_df)

    model = build_model(cfg).to(device)
    if cfg.init_from:
        ckpt = torch.load(cfg.init_from, map_location=device)
        model.load_state_dict(ckpt["model_state_dict"])
```

```
        print(f"Initialized weights from {cfg.init_from}")
    criterion = build_criterion(cfg)
    optimizer = build_optimizer(cfg, model)
    scheduler = build_scheduler(cfg, optimizer)
    use_amp = cfg.amp and device.type == "cuda"
    scaler = torch.amp.GradScaler("cuda", enabled=use_amp)
    ema = ModelEMA(model, cfg.ema_decay) if cfg.ema else None

    history = {
        "train_loss": [], "train_acc": [], "train_f1": [],
        "val_loss": [], "val_acc": [], "val_f1": [],
        "ema_val_loss": [], "ema_val_acc": [], "ema_val_f1": [],
        "lr": [], "epoch_time": [],
    }
    start_epoch = 0
    best_val_acc = 0.0
    best_val_f1 = 0.0
    best_epoch = -1
    best_used_ema = False

    last_path = os.path.join(run_dir, f"{tag}_last.pt")
    best_path = os.path.join(run_dir, f"{tag}_best.pt")
    metrics_path = os.path.join(run_dir, f"{tag}_metrics.jsonl")

    if resume and os.path.exists(last_path):
        ckpt = torch.load(last_path, map_location=device)
        model.load_state_dict(ckpt["model_state_dict"])
        optimizer.load_state_dict(ckpt["optimizer_state_dict"])
        if scheduler is not None and ckpt.get("scheduler_state_dict"):
            scheduler.load_state_dict(ckpt["scheduler_state_dict"])
        scaler.load_state_dict(ckpt["scaler_state_dict"])
        if ema is not None and ckpt.get("ema_state_dict"):
            ema.ema.load_state_dict(ckpt["ema_state_dict"])
        start_epoch = ckpt["epoch"]
        best_val_acc = ckpt.get("best_val_acc", 0.0)
        best_val_f1 = ckpt.get("best_val_f1", 0.0)
        best_epoch = ckpt.get("best_epoch", -1)
        best_used_ema = ckpt.get("best_used_ema", False)
        hist_path = os.path.join(run_dir, f"{tag}_history.json")
        if os.path.exists(hist_path):
            history = json.load(open(hist_path))
        print(f"Resumed {run_dir} at epoch {start_epoch}")

    print(f"Device: {device} | params: {model.num_parameters():,} | "
          f"train {len(train_df)} | val {len(val_df)} | {cfg.run_name}")
    print(f"Config: image_size={cfg.image_size} opt={cfg.opt} lr={cfg.lr} "
          f"loss={cfg.loss} ls={cfg.label_smoothing} bs={cfg.batch_size} "
```

```
          f"epochs={cfg.num_epochs} ema={cfg.ema}")

    for epoch in range(start_epoch, cfg.num_epochs):
        t0 = time.time()
        tr_loss, tr_acc, tr_f1 = train_epoch(
            model, train_loader, criterion, optimizer, scaler, device, cfg)
        val_loss, val_acc, val_f1 = evaluate(
            model, val_loader, criterion, device, cfg)

        ema_loss = ema_acc = ema_f1 = float("nan")
        if ema is not None:
            ema.update(model)
            ema_loss, ema_acc, ema_f1 = evaluate(
                ema.ema, val_loader, criterion, device, cfg)

        if scheduler is not None:
            scheduler.step()

        dt = time.time() - t0
        lr = optimizer.param_groups[0]["lr"]
        history["train_loss"].append(tr_loss)
        history["train_acc"].append(tr_acc)
        history["train_f1"].append(tr_f1)
        history["val_loss"].append(val_loss)
        history["val_acc"].append(val_acc)
        history["val_f1"].append(val_f1)
        history["ema_val_loss"].append(ema_loss)
        history["ema_val_acc"].append(ema_acc)
        history["ema_val_f1"].append(ema_f1)
        history["lr"].append(lr)
        history["epoch_time"].append(dt)

        score = ema_acc if (ema is not None and cfg.eval_ema) else val_acc
        if score > best_val_acc:
            best_val_acc = score
            best_val_f1 = ema_f1 if (ema is not None and cfg.eval_ema) else␣
↪val_f1
            best_epoch = epoch + 1
            best_used_ema = ema is not None and cfg.eval_ema
            best_model = ema.ema if best_used_ema else model
            torch.save({
                "model_state_dict": _state_to_cpu(best_model),
                "config": cfg_snapshot,
                "epoch": epoch + 1,
                "val_acc": score,
                "val_f1": best_val_f1,
                "used_ema": best_used_ema,
```

```
            }, best_path)

        if (epoch + 1) % cfg.log_interval == 0:
            print(f"Epoch {epoch + 1:3d}/{cfg.num_epochs} | "
                  f"train loss {tr_loss:.4f} acc {tr_acc:.4f} f1 {tr_f1:.4f} | "
                  f"val loss {val_loss:.4f} acc {val_acc:.4f} f1 {val_f1:.4f} |␣
↪"
                  f"ema acc {ema_acc:.4f} | lr {lr:.2e} | {dt:.1f}s",
                  flush=True)

        with open(metrics_path, "a") as f:
            f.write(json.dumps({
                "epoch": epoch + 1,
                "train_loss": tr_loss, "train_acc": tr_acc, "train_f1": tr_f1,
                "val_loss": val_loss, "val_acc": val_acc, "val_f1": val_f1,
                "ema_val_loss": ema_loss, "ema_val_acc": ema_acc,
                "ema_val_f1": ema_f1, "lr": lr, "epoch_time": dt,
            }) + "\n")

        torch.save({
            "model_state_dict": _state_to_cpu(model),
            "ema_state_dict": _state_to_cpu(ema.ema) if ema else None,
            "optimizer_state_dict": optimizer.state_dict(),
            "scheduler_state_dict": scheduler.state_dict() if scheduler else␣
↪None,
            "scaler_state_dict": scaler.state_dict(),
            "config": cfg_snapshot,
            "epoch": epoch + 1,
            "best_val_acc": best_val_acc,
            "best_val_f1": best_val_f1,
            "best_epoch": best_epoch,
            "best_used_ema": best_used_ema,
        }, last_path)

        with open(os.path.join(run_dir, f"{tag}_history.json"), "w") as f:
            json.dump(history, f, indent=2)

    results = {
        "run_name": cfg.run_name,
        "fold": cfg.val_fold,
        "best_val_acc": best_val_acc,
        "best_val_f1": best_val_f1,
        "final_val_acc": history["val_acc"][-1],
        "final_ema_val_acc": (history["ema_val_acc"][-1]
                              if history["ema_val_acc"] else None),
        "best_epoch": best_epoch,
        "used_ema": best_used_ema,
```

```
        "train_size": len(train_df),
        "val_size": len(val_df),
        "params": model.num_parameters(),
        "config": cfg_snapshot,
    }
    with open(os.path.join(run_dir, f"{tag}_results.json"), "w") as f:
        json.dump(results, f, indent=2)
    print(f"Done. Best val acc {best_val_acc:.4f} at epoch {best_epoch} "
          f"({'EMA' if best_used_ema else 'raw'})", flush=True)

    ckpt = torch.load(best_path, map_location=device)
    model.load_state_dict(ckpt["model_state_dict"])
    return model, history
```

[9]:
```
# EDIT THESE HYPERPARAMETERS AND RE-RUN THIS CELL + THE NEXT ONE
demo_cfg = Config()
demo_cfg.image_size = 64          # try 64 / 96 / 128
demo_cfg.num_epochs = 4           # try 1-5
demo_cfg.warmup_epochs = 1
demo_cfg.batch_size = 64
demo_cfg.lr = 1e-3
demo_cfg.opt = "adamw"            # adamw / adam / sgd / momentum / nag / gd
demo_cfg.activation = "relu"      # relu / gelu / silu / leaky_relu / ...
demo_cfg.label_smoothing = 0.1
demo_cfg.limit_per_class = 500    # small subset for the CPU demo
demo_cfg.num_workers = 0
demo_cfg.device = "cpu"
demo_cfg.ema = False              # too few steps for a useful EMA
demo_cfg.run_name = "demo"
demo_cfg.output_dir = "demo_runs"
demo_cfg
```

[9]:
```
Config(image_size=64, num_folds=5, val_fold=0, seed=42, split_seed=42,
num_workers=0, limit_per_class=500, num_hidden=4, sizes=[64, 128, 256, 512],
blocks_per_stage=[2, 2, 2, 2], activation='relu', stem_width=64, dropout=0.2,
loss='cross_entropy', label_smoothing=0.1, opt='adamw', lr=0.001, momentum=0.9,
weight_decay=0.0005, grad_clip=1.0, num_epochs=4, warmup_epochs=1, anneal=True,
min_lr_ratio=0.01, batch_size=64, amp=True, ema=False, ema_decay=0.999,
eval_ema=True, log_interval=1, aug_random_resized_crop=True, aug_scale_min=0.65,
aug_hflip=True, aug_rotation=7.0, aug_color_jitter=0.2, aug_random_erasing=0.25,
run_name='demo', device='cpu', output_dir='demo_runs', init_from='',
tta_views=4, predict_batch_size=192)
```

[10]:
```
if DATA_AVAILABLE:
    import shutil
    shutil.rmtree("demo_runs", ignore_errors=True)
    demo_model, demo_hist = fit(demo_cfg, TRAIN_CSV, TRAIN_DIR)
```

```
    print(f"Demo validation accuracy (last epoch): "
          f"{100 * demo_hist['val_acc'][-1]:.2f}%")
else:
    print("Dataset not found; skipping the live demo. Set GLASSES_DATA_ROOT "
          "to the folder containing kaggle/ and Dataset/ and re-run.")
```

```
Device: cpu | params: 11,177,538 | train 800 | val 200 | demo
Config: image_size=64 opt=adamw lr=0.001 loss=cross_entropy ls=0.1 bs=64
epochs=4 ema=False

Epoch   1/4 | train loss 0.6973 acc 0.5573 f1 0.5550 | val loss 0.6853 acc
0.5700 f1 0.3281 | ema acc nan | lr 1.00e-03 | 3.2s

Epoch   2/4 | train loss 0.6070 acc 0.6849 f1 0.6841 | val loss 0.7158 acc
0.6150 f1 0.4832 | ema acc nan | lr 7.52e-04 | 3.1s

Epoch   3/4 | train loss 0.4999 acc 0.7643 f1 0.7524 | val loss 0.7728 acc
0.5800 f1 0.3636 | ema acc nan | lr 2.58e-04 | 3.2s

Epoch   4/4 | train loss 0.3999 acc 0.8698 f1 0.8619 | val loss 0.6433 acc
0.7200 f1 0.6316 | ema acc nan | lr 1.00e-05 | 3.0s

Done. Best val acc 0.7200 at epoch 4 (raw)

Demo validation accuracy (last epoch): 72.00%
```

The demo's accuracy is intentionally modest (64px, 1,000 images, 4 epochs). It demonstrates that the hyperparameters and architecture are live; the submitted models use the full dataset at 224px for 90 epochs with EMA.

1.6 5. Hyperparameter sweep (160px, 30 epochs, fold 0)

Six configurations were compared. Every convolutional configuration is above 99.6% validation accuracy. Removing augmentation causes the largest drop; optimizer and label-smoothing differences are within 0.05%. The small 2.71M model wins this short sweep by 0.02%, which is within noise, so the medium ResNet-18-style model was selected for the longer 224px schedule.

[11]:
```
sweep_rows = [{'run': 'sweep_01_r18_adamw',
  'params_M': 11.18,
  'opt': 'adamw',
  'lr': 0.001,
  'label_smoothing': 0.1,
  'augmentation': True,
  'val_acc_pct': 99.9592,
  'val_f1_pct': 99.9593,
  'best_epoch': 25},
 {'run': 'sweep_02_r18_sgd',
  'params_M': 11.18,
  'opt': 'momentum',
  'lr': 0.1,
  'label_smoothing': 0.1,
```

```
  'augmentation': True,
  'val_acc_pct': 99.9388,
  'val_f1_pct': 99.9389,
  'best_epoch': 22},
 {'run': 'sweep_03_small_adamw',
  'params_M': 2.71,
  'opt': 'adamw',
  'lr': 0.001,
  'label_smoothing': 0.1,
  'augmentation': True,
  'val_acc_pct': 99.9796,
  'val_f1_pct': 99.9797,
  'best_epoch': 29},
 {'run': 'sweep_04_r34_adamw',
  'params_M': 21.29,
  'opt': 'adamw',
  'lr': 0.001,
  'label_smoothing': 0.1,
  'augmentation': True,
  'val_acc_pct': 99.9388,
  'val_f1_pct': 99.9389,
  'best_epoch': 26},
 {'run': 'sweep_05_r18_noaug',
  'params_M': 11.18,
  'opt': 'adamw',
  'lr': 0.001,
  'label_smoothing': 0.1,
  'augmentation': False,
  'val_acc_pct': 99.6735,
  'val_f1_pct': 99.6735,
  'best_epoch': 13},
 {'run': 'sweep_06_r18_nols',
  'params_M': 11.18,
  'opt': 'adamw',
  'lr': 0.001,
  'label_smoothing': 0.0,
  'augmentation': True,
  'val_acc_pct': 99.9388,
  'val_f1_pct': 99.9389,
  'best_epoch': 26}]

sweep_df = pd.DataFrame(sweep_rows).sort_values("val_acc_pct",
                                                ascending=False)
sweep_df.reset_index(drop=True)
```

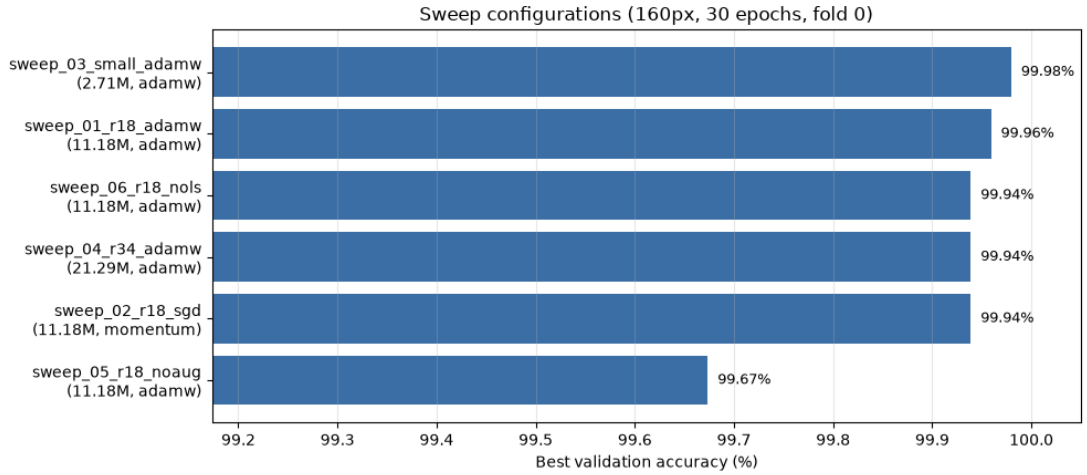
[11]:

```
                   run  params_M       opt     lr  label_smoothing  \
0  sweep_03_small_adamw      2.71     adamw  0.001              0.1
```

```
1    sweep_01_r18_adamw     11.18     adamw  0.001              0.1
2      sweep_02_r18_sgd     11.18  momentum  0.100              0.1
3    sweep_04_r34_adamw     21.29     adamw  0.001              0.1
4     sweep_06_r18_nols     11.18     adamw  0.001              0.0
5    sweep_05_r18_noaug     11.18     adamw  0.001              0.1
```

	augmentation	val_acc_pct	val_f1_pct	best_epoch
0	True	99.9796	99.9797	29
1	True	99.9592	99.9593	25
2	True	99.9388	99.9389	22
3	True	99.9388	99.9389	26
4	True	99.9388	99.9389	26
5	False	99.6735	99.6735	13

[12]:
```
rows = sorted(sweep_rows, key=lambda r: r["val_acc_pct"])
names = [f'{r["run"]}\n({r["params_M"]}M, {r["opt"]})' for r in rows]
vals = [r["val_acc_pct"] for r in rows]
plt.figure(figsize=(10, 4.5))
bars = plt.barh(names, vals, color="#3b6ea5")
for b, v in zip(bars, vals):
    plt.text(v + 0.01, b.get_y() + b.get_height() / 2,
             f"{v:.2f}%", va="center", fontsize=9)
plt.xlim(min(vals) - 0.5, 100.05)
plt.xlabel("Best validation accuracy (%)")
plt.title("Sweep configurations (160px, 30 epochs, fold 0)")
plt.grid(axis="x", alpha=0.3)
plt.tight_layout()
plt.show()
```



1.7 6. Five-fold cross-validation (224px, 90 epochs)

The selected recipe was trained on all five folds with a fixed data split and a different model-init seed per fold. The best epoch is selected on EMA validation accuracy.

[13]:
```
cv_rows = [{'fold': 0,
  'best_epoch': 63,
  'val_acc_pct': 99.9796,
  'val_f1_pct': 99.9797,
  'used_ema': True,
  'train_images': 19598,
  'val_images': 4901},
 {'fold': 1,
  'best_epoch': 90,
  'val_acc_pct': 99.9592,
  'val_f1_pct': 99.9593,
  'used_ema': True,
  'train_images': 19598,
  'val_images': 4901},
 {'fold': 2,
  'best_epoch': 51,
  'val_acc_pct': 99.9592,
  'val_f1_pct': 99.9593,
  'used_ema': True,
  'train_images': 19598,
  'val_images': 4901},
 {'fold': 3,
  'best_epoch': 27,
  'val_acc_pct': 99.9592,
  'val_f1_pct': 99.9593,
  'used_ema': True,
  'train_images': 19598,
  'val_images': 4901},
 {'fold': 4,
  'best_epoch': 22,
  'val_acc_pct': 99.8979,
  'val_f1_pct': 99.8981,
  'used_ema': True,
  'train_images': 19604,
  'val_images': 4895}]

cv_df = pd.DataFrame(cv_rows)
mean_acc = sum(r["val_acc_pct"] for r in cv_rows) / len(cv_rows)
mean_f1 = sum(r["val_f1_pct"] for r in cv_rows) / len(cv_rows)
std_acc = (sum((r["val_acc_pct"] - mean_acc) ** 2
               for r in cv_rows) / len(cv_rows)) ** 0.5
print(f"Mean validation accuracy: {mean_acc:.4f}% +/- {std_acc:.4f}%")
print(f"Mean validation F1:       {mean_f1:.4f}%")
```

```
cv_df
```

```
Mean validation accuracy: 99.9510% +/- 0.0277%
Mean validation F1:       99.9511%
```

[13]:

	fold	best_epoch	val_acc_pct	val_f1_pct	used_ema	train_images	val_images
0	0	63	99.9796	99.9797	True	19598	4901
1	1	90	99.9592	99.9593	True	19598	4901
2	2	51	99.9592	99.9593	True	19598	4901
3	3	27	99.9592	99.9593	True	19598	4901
4	4	22	99.8979	99.8981	True	19604	4895

[14]:
```
cv_histories = {0: {'val_acc_pct': [97.8372,
                    98.837,
                    91.4915,
                    98.3065,
                    99.4083,
                    99.6735,
                    99.2246,
                    99.4287,
                    99.1226,
                    99.4083,
                    99.6735,
                    99.5511,
                    99.796,
                    99.2246,
                    99.7756,
                    99.4899,
                    99.6531,
                    99.8776,
                    99.8572,
                    97.2251,
                    99.8776,
                    99.9184,
                    99.796,
                    99.7347,
                    99.6735,
                    99.8776,
                    99.8572,
                    99.8776,
```

```
                    99.8776,
                    99.9388,
                    99.898,
                    99.8776,
                    99.8572,
                    99.6123,
                    99.9592,
                    99.8776,
                    99.9184,
                    99.9592,
                    99.9184,
                    99.9388,
                    98.8982,
                    99.9592,
                    99.9184,
                    99.9796,
                    99.898,
                    99.9592,
                    99.8572,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9388,
                    99.9184,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.898,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9184,
                    99.9796,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9592,
                    99.9388,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9388,
                    99.9592,
```

```
                    99.9592,
                    99.9388,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592],
    'ema_val_acc_pct': [97.6331,
                        97.5515,
                        96.8986,
                        98.3065,
                        84.1665,
                        99.7143,
                        99.1022,
                        99.6735,
                        99.8368,
                        98.2657,
                        96.2049,
                        99.6939,
                        99.7143,
                        99.1226,
                        98.5717,
                        99.2246,
                        99.8164,
                        99.7143,
                        97.9596,
                        99.4695,
                        99.3675,
                        99.6939,
                        99.7347,
                        99.5919,
                        99.7347,
                        99.9184,
                        99.9184,
                        99.7552,
                        99.898,
                        99.5919,
                        99.8368,
                        99.8164,
```

```
                    99.5511,
                    99.3471,
                    99.6735,
                    99.7143,
                    99.7756,
                    99.9388,
                    99.7552,
                    99.8572,
                    99.6735,
                    99.7552,
                    99.9184,
                    99.9388,
                    99.796,
                    99.9388,
                    99.9184,
                    99.9592,
                    99.9388,
                    99.9388,
                    99.9592,
                    99.898,
                    99.9184,
                    99.9388,
                    99.7756,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9796,
                    99.9592,
                    99.9592,
                    99.9388,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9592,
```

```
                        99.9388,
                        99.9592,
                        99.9592,
                        99.9796,
                        99.9796,
                        99.9796,
                        99.9592,
                        99.9592,
                        99.9592,
                        99.9592,
                        99.9592]},
 1: {'val_acc_pct': [99.6123,
                     99.3063,
                     99.0206,
                     99.7143,
                     99.7347,
                     99.2655,
                     98.5717,
                     99.3471,
                     99.3879,
                     97.5107,
                     99.8776,
                     99.7756,
                     99.6939,
                     99.7756,
                     99.8164,
                     99.7347,
                     99.3675,
                     99.3267,
                     99.4695,
                     99.8164,
                     99.2042,
                     99.8368,
                     99.8368,
                     99.9184,
                     99.4491,
                     99.7552,
                     99.898,
                     99.7756,
                     99.8776,
                     99.9184,
                     99.8164,
                     99.8572,
                     99.796,
                     99.8572,
                     99.9388,
                     99.9184,
```

```
                    99.9184,
                    99.9184,
                    99.9184,
                    99.898,
                    99.8776,
                    99.8776,
                    99.898,
                    99.8776,
                    99.796,
                    99.8164,
                    99.898,
                    99.9184,
                    99.9184,
                    99.8572,
                    99.9184,
                    99.9184,
                    99.8776,
                    99.9184,
                    99.8776,
                    99.8776,
                    99.9592,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9184,
                    99.9388,
                    99.9388,
                    99.9184,
                    99.9388,
                    99.9388,
                    99.9184,
                    99.9592,
                    99.9592,
                    99.898,
                    99.9388,
                    99.9592,
                    99.9388,
                    99.9388,
                    99.9592,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9592,
                    99.9388,
                    99.9388,
                    99.9592,
```

```
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9592,
                    99.9388,
                    99.9592,
                    99.9592],
    'ema_val_acc_pct': [89.0839,
                        98.5513,
                        91.7568,
                        96.6741,
                        98.4493,
                        97.1026,
                        98.7145,
                        99.2451,
                        99.4899,
                        88.8798,
                        99.041,
                        98.3065,
                        99.7552,
                        95.3275,
                        99.5103,
                        98.6533,
                        99.041,
                        99.0206,
                        96.4497,
                        99.7143,
                        99.5103,
                        99.796,
                        99.8572,
                        99.8572,
                        99.8776,
                        98.3677,
                        99.796,
                        98.7145,
                        97.9596,
                        99.898,
                        99.8368,
                        99.8164,
                        99.8368,
                        99.898,
                        99.8164,
                        99.9184,
                        99.7347,
                        99.6939,
                        99.8776,
                        99.7552,
```

```
                    99.8164,
                    99.6735,
                    99.8572,
                    99.9388,
                    99.9184,
                    99.7143,
                    99.6939,
                    99.7756,
                    99.8368,
                    99.8572,
                    99.8368,
                    99.9388,
                    99.9184,
                    99.9184,
                    99.9184,
                    99.898,
                    99.9184,
                    99.9184,
                    99.9184,
                    99.9388,
                    99.9388,
                    99.8776,
                    99.898,
                    99.9184,
                    99.9388,
                    99.9388,
                    99.9184,
                    99.9388,
                    99.9184,
                    99.898,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9184,
                    99.9388,
                    99.9184,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9388,
```

```
                        99.9388,
                        99.9388,
                        99.9592]},
 2: {'val_acc_pct': [99.3879,
                     99.0614,
                     93.1647,
                     94.6337,
                     98.5717,
                     99.6123,
                     99.4899,
                     99.6939,
                     99.6939,
                     99.7347,
                     98.6329,
                     99.0614,
                     99.7552,
                     99.6123,
                     99.3675,
                     99.6735,
                     99.796,
                     99.9184,
                     99.2451,
                     99.0002,
                     99.8572,
                     99.796,
                     98.8982,
                     99.796,
                     99.7552,
                     99.8368,
                     99.7756,
                     99.6735,
                     99.9184,
                     99.9388,
                     99.796,
                     99.8164,
                     99.9184,
                     96.6537,
                     99.8572,
                     99.9184,
                     99.9184,
                     99.8776,
                     99.898,
                     99.8572,
                     99.7552,
                     99.9388,
                     99.8572,
                     99.8164,
```

```
                    99.9184,
                    99.8776,
                    99.8776,
                    99.9388,
                    99.8572,
                    99.9184,
                    99.9388,
                    99.9184,
                    99.898,
                    99.8572,
                    99.898,
                    99.8572,
                    99.9184,
                    99.8776,
                    99.9184,
                    99.9592,
                    99.898,
                    99.9184,
                    99.9388,
                    99.9388,
                    99.8776,
                    99.9184,
                    99.898,
                    99.8776,
                    99.898,
                    99.8572,
                    99.898,
                    99.8776,
                    99.898,
                    99.8776,
                    99.898,
                    99.9388,
                    99.9184,
                    99.9184,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9184,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9184,
                    99.9388,
                    99.9184,
                    99.898],
    'ema_val_acc_pct': [99.041,
```

```
                    97.9392,
                    97.8576,
                    94.0624,
                    78.4126,
                    98.939,
                    95.8172,
                    99.6123,
                    96.9598,
                    99.796,
                    99.1838,
                    99.5103,
                    99.6327,
                    98.6329,
                    99.6327,
                    98.735,
                    99.6531,
                    99.8368,
                    99.6939,
                    98.6533,
                    99.7756,
                    99.8572,
                    99.8368,
                    99.4899,
                    99.6735,
                    99.7347,
                    99.6939,
                    99.5307,
                    99.5103,
                    99.796,
                    99.8368,
                    99.7756,
                    99.6123,
                    99.5715,
                    99.898,
                    99.898,
                    99.898,
                    99.898,
                    99.9184,
                    99.9388,
                    99.898,
                    99.898,
                    99.9388,
                    99.898,
                    99.7347,
                    99.9184,
                    99.9184,
                    99.9184,
```

```
                        99.8368,
                        99.9184,
                        99.9592,
                        99.9592,
                        99.9184,
                        99.9388,
                        99.8368,
                        99.9592,
                        99.9388,
                        99.9184,
                        99.898,
                        99.8368,
                        99.8164,
                        99.8164,
                        99.7756,
                        99.8776,
                        99.8572,
                        99.898,
                        99.9388,
                        99.8368,
                        99.8164,
                        99.8368,
                        99.8572,
                        99.8368,
                        99.8776,
                        99.8776,
                        99.8572,
                        99.898,
                        99.898,
                        99.9184,
                        99.898,
                        99.9184,
                        99.898,
                        99.8572,
                        99.9184,
                        99.9184,
                        99.9184,
                        99.898,
                        99.898,
                        99.9184,
                        99.9388,
                        99.9388]},
 3: {'val_acc_pct': [99.143,
                     98.939,
                     99.5103,
                     97.8372,
                     99.041,
```

```
                    96.7762,
                    99.6531,
                    99.1838,
                    99.8368,
                    99.4491,
                    99.7552,
                    99.6327,
                    99.3675,
                    99.6735,
                    99.7756,
                    99.8572,
                    99.898,
                    99.8572,
                    99.898,
                    99.6939,
                    99.9184,
                    99.4695,
                    99.7347,
                    99.6735,
                    99.796,
                    99.8776,
                    99.9388,
                    99.898,
                    99.9184,
                    99.8776,
                    99.9184,
                    99.8368,
                    99.9592,
                    99.898,
                    99.8776,
                    99.898,
                    99.898,
                    99.9592,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9592,
                    99.9388,
                    99.9184,
                    99.8164,
                    99.9184,
                    99.9592,
                    99.898,
                    99.8572,
                    99.9388,
                    99.9388,
                    99.9184,
```

```
                    99.9184,
                    99.9592,
                    99.9388,
                    99.898,
                    99.9184,
                    99.9184,
                    99.898,
                    99.9184,
                    99.9388,
                    99.898,
                    99.7756,
                    99.9592,
                    99.898,
                    99.9184,
                    99.9184,
                    99.898,
                    99.8776,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9592,
                    99.9592,
                    99.9388,
                    99.9388,
                    99.9592,
                    99.9388,
                    99.9592,
                    99.9388,
                    99.9388,
                    99.9388,
                    99.9592,
                    99.9592,
                    99.9592],
    'ema_val_acc_pct': [90.3489,
                        97.6739,
                        99.2246,
                        98.7962,
                        86.6558,
                        61.008,
                        98.939,
                        98.0412,
                        98.7554,
```

```
                    99.7552,
                    99.7347,
                    99.8776,
                    99.7552,
                    98.4697,
                    99.8776,
                    99.8164,
                    99.4899,
                    99.796,
                    99.0206,
                    99.9184,
                    99.8368,
                    99.4899,
                    99.9184,
                    99.7347,
                    99.8368,
                    99.7143,
                    99.9592,
                    98.7554,
                    99.6123,
                    99.7552,
                    99.9184,
                    99.898,
                    99.7552,
                    99.5715,
                    99.3675,
                    99.796,
                    99.9184,
                    99.9388,
                    99.8776,
                    99.898,
                    99.8776,
                    99.9184,
                    99.8776,
                    99.9592,
                    99.9592,
                    99.898,
                    99.9388,
                    99.9592,
                    99.9184,
                    99.898,
                    99.9592,
                    99.8368,
                    99.9592,
                    99.9388,
                    99.8572,
                    99.9184,
```

```
                        99.9184,
                        99.9184,
                        99.9388,
                        99.8572,
                        99.9388,
                        99.9388,
                        99.9184,
                        99.9388,
                        99.9184,
                        99.796,
                        99.8776,
                        99.9592,
                        99.898,
                        99.9184,
                        99.9184,
                        99.898,
                        99.9184,
                        99.9184,
                        99.9388,
                        99.9388,
                        99.9388,
                        99.9388,
                        99.9184,
                        99.9388,
                        99.9388,
                        99.9388,
                        99.9388,
                        99.9388,
                        99.9388,
                        99.9388,
                        99.9388,
                        99.9388,
                        99.9388,
                        99.9388,
                        99.9388]},
 4: {'val_acc_pct': [98.8764,
                     98.3453,
                     99.1011,
                     98.856,
                     98.5904,
                     99.6731,
                     99.4484,
                     99.4076,
                     98.713,
                     99.571,
                     99.0807,
                     99.428,
                     99.6527,
```

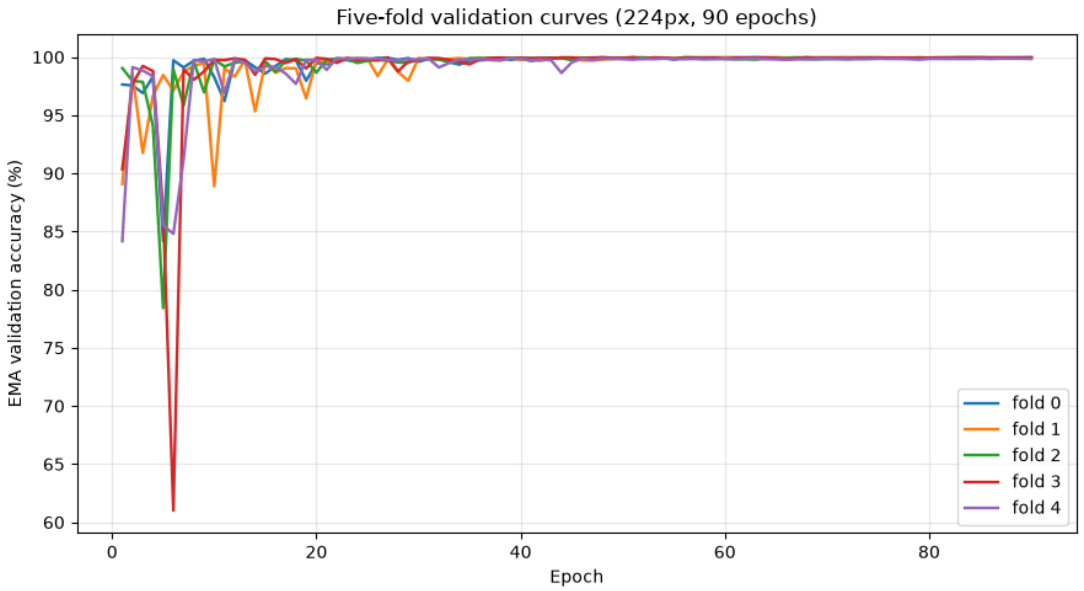
```
                    99.5097,
                    98.9173,
                    99.3871,
                    99.857,
                    96.2615,
                    95.0358,
                    99.714,
                    99.6118,
                    99.7753,
                    99.714,
                    99.7549,
                    99.7344,
                    99.7753,
                    99.8366,
                    99.7344,
                    99.4893,
                    99.7957,
                    99.8366,
                    99.7957,
                    99.7549,
                    99.857,
                    99.857,
                    99.8161,
                    99.8366,
                    99.7957,
                    99.9183,
                    99.7957,
                    99.857,
                    99.8366,
                    99.857,
                    99.8366,
                    99.8161,
                    99.7753,
                    99.7957,
                    99.8774,
                    99.8161,
                    99.8366,
                    99.8366,
                    99.8366,
                    99.8774,
                    99.857,
                    99.8366,
                    99.8366,
                    99.7753,
                    99.7753,
                    99.8366,
                    99.8161,
```

```
                    99.7957,
                    99.7957,
                    99.8161,
                    99.8366,
                    99.8366,
                    99.8366,
                    99.7957,
                    99.8161,
                    99.8366,
                    99.8366,
                    99.8161,
                    99.8161,
                    99.857,
                    99.857,
                    99.857,
                    99.8366,
                    99.8366,
                    99.8774,
                    99.857,
                    99.8774,
                    99.8774,
                    99.8774,
                    99.8774,
                    99.8774,
                    99.8774,
                    99.8774,
                    99.8774,
                    99.8774,
                    99.8774,
                    99.8774],
    'ema_val_acc_pct': [84.1675,
                        99.1216,
                        98.8355,
                        98.3453,
                        85.5158,
                        84.8008,
                        91.1951,
                        99.7344,
                        99.6323,
                        99.8161,
                        96.8948,
                        99.7957,
                        99.5097,
                        98.7538,
                        99.2033,
                        99.142,
                        98.57,
```

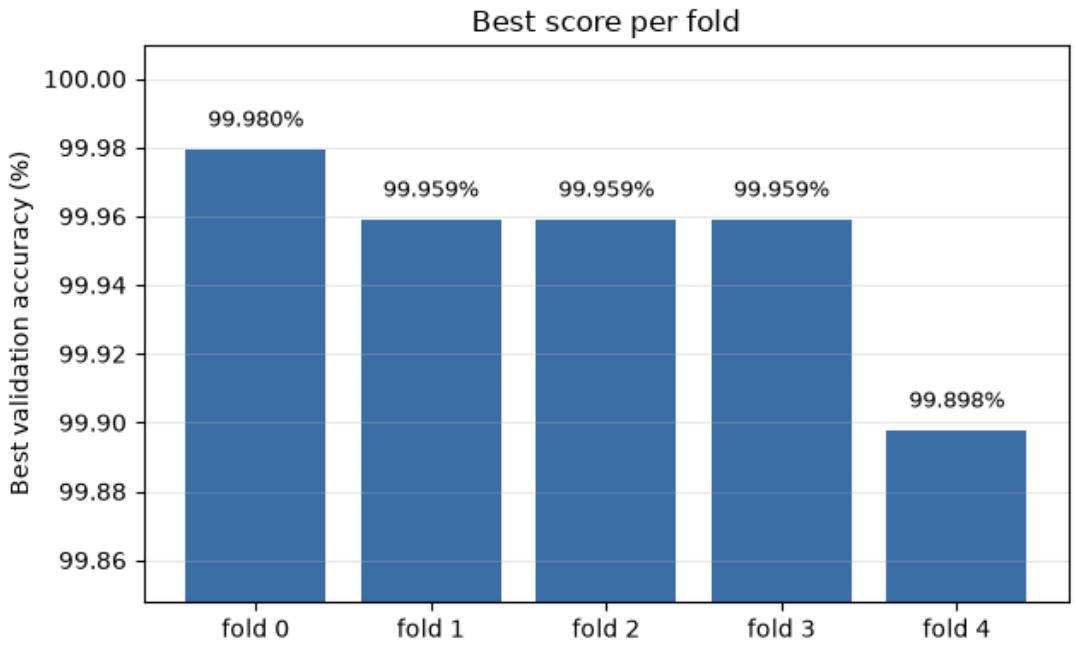
```
                    97.6915,
                    99.7549,
                    99.7549,
                    98.8968,
                    99.8979,
                    99.8366,
                    99.8979,
                    99.857,
                    99.8774,
                    99.7344,
                    99.714,
                    99.7957,
                    99.7957,
                    99.857,
                    99.1011,
                    99.4893,
                    99.7753,
                    99.7753,
                    99.8161,
                    99.7549,
                    99.7344,
                    99.8774,
                    99.857,
                    99.714,
                    99.7957,
                    99.714,
                    98.6313,
                    99.5301,
                    99.857,
                    99.7344,
                    99.857,
                    99.857,
                    99.8161,
                    99.7753,
                    99.8366,
                    99.8161,
                    99.857,
                    99.8161,
                    99.8161,
                    99.8366,
                    99.7957,
                    99.8161,
                    99.8366,
                    99.8161,
                    99.8366,
                    99.857,
                    99.8161,
```

```
                        99.7957,
                        99.7549,
                        99.7957,
                        99.7957,
                        99.8161,
                        99.8366,
                        99.8161,
                        99.7957,
                        99.8161,
                        99.8161,
                        99.857,
                        99.8366,
                        99.8366,
                        99.8161,
                        99.7753,
                        99.8366,
                        99.8366,
                        99.8366,
                        99.8366,
                        99.8366,
                        99.857,
                        99.8366,
                        99.857,
                        99.857,
                        99.857,
                        99.8366]}}

plt.figure(figsize=(9, 5))
for fold, h in cv_histories.items():
    y = h["ema_val_acc_pct"] or h["val_acc_pct"]
    plt.plot(range(1, len(y) + 1), y, label=f"fold {fold}", lw=1.7)
plt.xlabel("Epoch")
plt.ylabel("EMA validation accuracy (%)")
plt.title("Five-fold validation curves (224px, 90 epochs)")
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```



[15]:
```
fig, ax = plt.subplots(figsize=(6.5, 4))
folds = [f'fold {r["fold"]}' for r in cv_rows]
accs = [r["val_acc_pct"] for r in cv_rows]
bars = ax.bar(folds, accs, color="#3b6ea5")
for b, v in zip(bars, accs):
    ax.text(b.get_x() + b.get_width() / 2, v + 0.005, f"{v:.3f}%",
            ha="center", va="bottom", fontsize=9)
ax.set_ylim(min(accs) - 0.05, max(accs) + 0.03)
ax.set_ylabel("Best validation accuracy (%)")
ax.set_title("Best score per fold")
ax.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()
```



1.8 7. Test-time augmentation and ensemble inference

Each of the five fold models predicts the test set under four deterministic views: identity, horizontal flip, and two zoomed (and horizontally flipped) views. Softmax probabilities are averaged within a model and then across the five models. The final label is the argmax of the averaged probability.

The code below is the exact inference routine. It is defined here so the notebook is self-contained; set `RUN_ENSEMBLE = True` and point to the five checkpoints (in the repository) to regenerate `submission.csv`.

[16]:
```
def make_views(size, n_views=4):
    """Deterministic TTA views: identity, hflip, and zoomed variants."""
    def view(resize_to, flip):
        ops = [T.Resize(resize_to), T.CenterCrop(size)]
        if flip:
            ops.append(T.RandomHorizontalFlip(p=1.0))
        ops += [T.ToTensor(), T.Normalize(MEAN, STD)]
        return T.Compose(ops)

    all_views = [
        view(size + 32, False),
        view(size + 32, True),
        view(int(round(size * 1.15)) + 32, False),
        view(int(round(size * 1.15)) + 32, True),
        view(size + 96, False),
        view(size + 96, True),
```

```
    ]
    return all_views[:n_views]


@torch.no_grad()
def predict_probs(model, loader, device, use_amp):
    model.eval()
    chunks = []
    for images, _ in loader:
        images = images.to(device, non_blocking=True)
        with torch.amp.autocast("cuda", enabled=use_amp):
            logits = model(images)
        chunks.append(torch.softmax(logits.float(), dim=1))
    return torch.cat(chunks)


def build_test_loader(test_csv, test_dir, transform, batch_size=128,
                      num_workers=0):
    df = pd.read_csv(test_csv)
    ds = GlassesDataset(df, test_dir, transform, is_test=True)
    loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
                        num_workers=num_workers,
                        pin_memory=torch.cuda.is_available())
    return loader, df


def run_ensemble(checkpoints, test_csv, test_dir, output_csv,
                 tta_views=4, batch_size=128, num_workers=0):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    use_amp = device.type == "cuda"
    per_model = []
    for path in checkpoints:
        ckpt = torch.load(path, map_location="cpu")
        d = ckpt["config"]
        cfg_m = Config(**{k: v for k, v in d.items()
                          if k in Config.__dataclass_fields__})
        model = build_model(cfg_m)
        model.load_state_dict(ckpt["model_state_dict"])
        model.to(device).eval()
        views = make_views(cfg_m.image_size, tta_views)
        acc = None
        for transform in views:
            loader, _ = build_test_loader(
                test_csv, test_dir, transform, batch_size, num_workers)
            probs = predict_probs(model, loader, device, use_amp)
            acc = probs if acc is None else acc + probs
        per_model.append((acc / len(views)).cpu())
```

```
        torch.cuda.empty_cache()
    stacked = torch.stack(per_model)
    ensemble_probs = stacked.mean(dim=0)
    preds = ensemble_probs.argmax(dim=1).tolist()
    df = pd.read_csv(test_csv)
    pd.DataFrame({"id": df["id"].astype(str),
                  "glasses": preds}).to_csv(output_csv, index=False)
    print(f"Saved {output_csv}: {len(preds)} rows, "
          f"{sum(preds)} glasses, {len(preds) - sum(preds)} no-glasses")
    return ensemble_probs, preds
```

[17]:
```
ensemble_summary = {'checkpoints': ['fold0_best.pt', 'fold1_best.pt',␣
 ↪'fold2_best.pt', 'fold3_best.pt', 'fold4_best.pt'],
 'tta_views': 4,
 'image_size': 224,
 'n_test': 5412,
 'glasses': 2984,
 'no_glasses': 2428,
 'mean_confidence': 0.9526850581169128,
 'unanimous': 5378,
 'model_disagreements': 34,
 'val_accs': [0.9997959600081616,
              0.9995919200163232,
              0.9995919200163232,
              0.9995919200163232,
              0.9989785495403473],
 'kaggle_score': 0.99872,
 'note': '5 fold models x 4 deterministic TTA views; probability probability␣
 ↪averaging'}

summary_df = pd.DataFrame([{
    "test images": ensemble_summary["n_test"],
    "glasses": ensemble_summary["glasses"],
    "no glasses": ensemble_summary["no_glasses"],
    "unanimous": ensemble_summary["unanimous"],
    "model disagreements": ensemble_summary["model_disagreements"],
    "mean confidence": round(ensemble_summary["mean_confidence"], 4),
    "Test score": ensemble_summary["kaggle_score"],
}])
summary_df
```

[17]:
```
   test images  glasses  no glasses  unanimous  model disagreements  \
0         5412     2984        2428       5378                   34

   mean confidence  Test score
0           0.9527     0.99872
```

[18]:
```
RUN_ENSEMBLE = False
if RUN_ENSEMBLE:
    checkpoints = [f"../experiments/cv_final/fold{i}_best.pt"
                   for i in range(5)]
    run_ensemble(checkpoints, TEST_CSV, TEST_DIR,
                 "submission_selfcontained.csv", tta_views=4,
                 batch_size=128, num_workers=0)
else:
    print("Set RUN_ENSEMBLE = True and provide the five fold checkpoints "
          "from the repository to regenerate the submission.")
```

```
Set RUN_ENSEMBLE = True and provide the five fold checkpoints from the
repository to regenerate the submission.
```

[19]:
```
submission_preview = [{'id': '1', 'glasses': 0},
 {'id': '2', 'glasses': 0},
 {'id': '3', 'glasses': 1},
 {'id': '4', 'glasses': 0},
 {'id': '5', 'glasses': 1},
 {'id': '6', 'glasses': 0},
 {'id': '7', 'glasses': 1},
 {'id': '8', 'glasses': 0},
 {'id': '9', 'glasses': 0},
 {'id': '10', 'glasses': 0}]
submission_counts = {'rows': 5412, 'glasses': 2984, 'no_glasses': 2428}

print(f"Submission: {submission_counts['rows']} rows | "
      f"{submission_counts['glasses']} glasses | "
      f"{submission_counts['no_glasses']} no-glasses")
pd.DataFrame(submission_preview)
```

```
Submission: 5412 rows | 2984 glasses | 2428 no-glasses
```

[19]:

	id	glasses
0	1	0
1	2	0
2	3	1
3	4	0
4	5	1
5	6	0
6	7	1
7	8	0
8	9	0
9	10	0

1.9 8. Results and discussion

The convolutional inductive bias, not external knowledge, accounts for the large improvement over the fully-connected baseline: local receptive fields and translation equivariance match the structure of the task. Strong augmentation, label smoothing, warmup + cosine scheduling, AMP and EMA stabilize training. Five-fold ensembling with TTA reduces prediction variance: only 34 of 5,412 test images have any model disagreement, and 5,378 are unanimous across all five models.

[20]:
```
comparison = [
    {"model": "MLP (earlier baseline)", "input": "64x64",
     "params_M": 6.46, "val_acc_pct": 86.69},
    {"model": "ResNet-style CNN, 5-fold mean", "input": "224x224",
     "params_M": 11.18, "val_acc_pct": 99.9510},
]
pd.DataFrame(comparison)
```

[20]:

	model	input	params_M	val_acc_pct
0	MLP (earlier baseline)	64x64	6.46	86.690
1	ResNet-style CNN, 5-fold mean	224x224	11.18	99.951

Final submission. `submission.csv` has 5,412 rows and the columns `id,glasses`. The labels are the argmax of the averaged neural-network softmax probabilities. Test score: **0.99872** (approximately 7 errors out of 5,412). The final labels are the argmax of the averaged network probabilities.

1.10 9. Reproducibility

- **Tunable hyperparameters.** `Config` and the live demo cell expose architecture, optimizer, loss, schedule, batch size, augmentation and annealing settings.
- **Repository:** https://github.com/D3LET3-57/GlassesNet
- **Report:** `report.pdf`.

Reproducing the final models. The final runs used 224px, 90 epochs, batch 128, AdamW at 1e-3, EMA 0.999 and the same five-fold split. On one RTX 6000 Ada this took about 2 hours for all five folds. The exact command in the repository is:

```
python -m src.run --config experiments/configs/final.json --fold 0 --run-name cv_final
```

The notebook's `fit` function is the same implementation, so you can also run `fit(Config(num_epochs=90, image_size=224, batch_size=128, num_workers=12, device='cuda'), TRAIN_CSV, TRAIN_DIR)` for each fold.

Additional ablation. A 288px fine-tune of the five fold models matched the 224px cross-validation mean exactly (99.9510%); it changed only 4 of 5,412 test predictions, fixing two and breaking two, so it was not used. The submitted ensemble is the 224px five-fold model.